

2D FSAE simulator

Generated by Doxygen 1.9.1

1 2D FSAE Simulation	1
1.1 Project Overview	1
1.2 Key Features	1
1.3 Project Structure	1
1.4 Global Call Graph	2
1.5 Task Descriptions	2
1.6 Building and Running the Simulation	2
2 Module Index	3
2.1 Modules	3
3 Class Index	5
3.1 Class List	5
4 File Index	7
4.1 File List	7
5 Module Documentation	9
5.1 Cone Detection and Mapping	9
5.1.1 Detailed Description	10
5.1.2 Function Documentation	10
5.1.2.1 calculate_circle_points()	10
5.1.2.2 check_nearest_point()	10
5.1.2.3 find_closest_points()	11
5.1.2.4 find_cone_center()	11
5.1.2.5 find_local_minima()	11
5.1.2.6 init_cone_borders()	11
5.1.2.7 lidar()	11
5.1.2.8 mapping()	12
5.1.2.9 update_map()	12
5.1.3 Variable Documentation	13
5.1.3.1 angle_step	13
5.1.3.2 candidates	13
5.1.3.3 detected_cones	13
5.1.3.4 distance_resolution	13
5.1.3.5 ignore_distance	13
5.1.3.6 n_candidates	13
5.1.3.7 sliding_window	13
5.1.3.8 start_angle	14
5.1.3.9 track_map	14
5.1.3.10 track_map_idx	14
5.2 Cone Detection and Mapping	14
5.3 Vehicle model and control	14
5.3.1 Detailed Description	14

5.3.2 Function Documentation	14
5.3.2.1 check_collisions()	15
5.3.2.2 vehicle_model()	15
6 Class Documentation	17
6.1 Button Struct Reference	17
6.1.1 Member Data Documentation	17
6.1.1.1 height	17
6.1.1.2 text	17
6.1.1.3 value	18
6.1.1.4 width	18
6.1.1.5 x	18
6.1.1.6 y	18
6.2 candidate_cone Struct Reference	18
6.2.1 Detailed Description	19
6.2.2 Member Data Documentation	19
6.2.2.1 color	19
6.2.2.2 detections	19
6.2.2.3 x	19
6.2.2.4 y	19
6.3 cone Struct Reference	20
6.3.1 Detailed Description	20
6.3.2 Member Data Documentation	20
6.3.2.1 color	20
6.3.2.2 x	20
6.3.2.3 y	21
6.4 cone_border Struct Reference	21
6.4.1 Detailed Description	21
6.4.2 Member Data Documentation	21
6.4.2.1 angles	21
6.4.2.2 color	22
6.5 Hough_circle_point_t Struct Reference	22
6.5.1 Detailed Description	22
6.5.2 Member Data Documentation	22
6.5.2.1 color	22
6.5.2.2 distance	22
6.5.2.3 x	23
6.5.2.4 y	23
6.6 pointcloud_t Struct Reference	23
6.6.1 Detailed Description	23
6.6.2 Member Data Documentation	23
6.6.2.1 color	23

6.6.2.2 distance	24
6.6.2.3 point_x	24
6.6.2.4 point_y	24
6.7 task_par Struct Reference	24
6.7.1 Member Data Documentation	25
6.7.1.1 arg	25
6.7.1.2 asem	25
6.7.1.3 at	25
6.7.1.4 deadline	25
6.7.1.5 dl	25
6.7.1.6 dmiss	25
6.7.1.7 period	25
6.7.1.8 prio	26
6.7.1.9 tid	26
6.8 timespec_custom Struct Reference	26
6.8.1 Member Data Documentation	26
6.8.1.1 tv_nsec	26
6.8.1.2 tv_sec	26
6.9 waypoint Struct Reference	27
6.9.1 Detailed Description	27
6.9.2 Member Data Documentation	27
6.9.2.1 x	27
6.9.2.2 y	27
7 File Documentation	29
7.1 include/control.h File Reference	29
7.1.1 Detailed Description	30
7.1.2 Function Documentation	31
7.1.2.1 autonomous_control()	31
7.1.2.2 keyboard_control()	32
7.1.3 Variable Documentation	33
7.1.3.1 pedal	33
7.1.3.2 steering	33
7.2 include/display.h File Reference	34
7.2.1 Detailed Description	35
7.2.2 Macro Definition Documentation	35
7.2.2.1 num_buttons	35
7.2.3 Function Documentation	36
7.2.3.1 draw_car()	36
7.2.3.2 draw_cone_map()	37
7.2.3.3 draw_detected_cones()	38
7.2.3.4 draw_dir_arrow()	38

7.2.3.5 draw_lidar()	39
7.2.3.6 draw_perception()	39
7.2.3.7 draw_track()	40
7.2.3.8 draw_trajectory()	41
7.2.3.9 update_display()	41
7.2.4 Variable Documentation	42
7.2.4.1 btn_state_autonomous	42
7.2.4.2 btn_state_cones	42
7.2.4.3 btn_state_map	43
7.2.4.4 btn_state_perception	43
7.2.4.5 btn_state_traj	43
7.3 include/globals.h File Reference	43
7.3.1 Detailed Description	45
7.3.2 Tasks Constants	45
7.3.3 Drawing Mutex	45
7.3.4 Conversion Constants	45
7.3.5 Visualization Constants	46
7.3.6 Car Pose	46
7.3.7 Perception Constants	46
7.3.8 Macro Definition Documentation	46
7.3.8.1 CONTROL_DEADLINE	47
7.3.8.2 CONTROL_PERIOD	47
7.3.8.3 CONTROL_PRIORITY	47
7.3.8.4 deg2rad	47
7.3.8.5 DISPLAY_DEADLINE	47
7.3.8.6 DISPLAY_PERIOD	47
7.3.8.7 DISPLAY_PRIORITY	47
7.3.8.8 MAX_CONES_MAP	47
7.3.8.9 MAX_DETECTED_CONES	48
7.3.8.10 maxThrottleHeight	48
7.3.8.11 PERCEPTION_DEADLINE	48
7.3.8.12 PERCEPTION_PERIOD	48
7.3.8.13 PERCEPTION_PRIORITY	48
7.3.8.14 PROFILING	48
7.3.8.15 px_per_meter	48
7.3.8.16 SETTINGS_DEADLINE	48
7.3.8.17 SETTINGS_PERIOD	49
7.3.8.18 SETTINGS_PRIORITY	49
7.3.8.19 TRAJECTORY_DEADLINE	49
7.3.8.20 TRAJECTORY_PERIOD	49
7.3.8.21 TRAJECTORY_PRIORITY	49
7.3.9 Variable Documentation	49

7.3.9.1 asphalt_gray	49
7.3.9.2 background	49
7.3.9.3 blue	50
7.3.9.4 car	50
7.3.9.5 car_angle	50
7.3.9.6 car_x	50
7.3.9.7 car_y	51
7.3.9.8 cone_radius	51
7.3.9.9 cones	51
7.3.9.10 control_panel	51
7.3.9.11 display_buffer	51
7.3.9.12 draw_mutex	52
7.3.9.13 grass_green	52
7.3.9.14 green	52
7.3.9.15 lidar_sem	52
7.3.9.16 measures	52
7.3.9.17 perception	53
7.3.9.18 pink	53
7.3.9.19 red	53
7.3.9.20 starting_cone_positions	53
7.3.9.21 steering_wheel	53
7.3.9.22 throttle_gauge	53
7.3.9.23 title	54
7.3.9.24 track	54
7.3.9.25 trajectory_bmp	54
7.3.9.26 white	54
7.3.9.27 X_MAX	54
7.3.9.28 Y_MAX	55
7.3.9.29 yellow	55
7.4 include/perception.h File Reference	55
7.4.1 Detailed Description	56
7.4.2 Macro Definition Documentation	57
7.4.2.1 DETECTIONS_THRESHOLD	57
7.4.2.2 MAX_CANDIDATES	57
7.4.2.3 MAX_POINTS_PER_CONE	57
7.4.2.4 maxRange	58
7.5 include/ptask.h File Reference	58
7.5.1 Detailed Description	60
7.5.2 Macro Definition Documentation	62
7.5.2.1 _POSIX_C_SOURCE	62
7.5.2.2 ACT	62
7.5.2.3 DEACT	62

7.5.2.4 MAX_TASKS	62
7.5.2.5 MICRO	62
7.5.2.6 NANO	62
7.5.3 Typedef Documentation	62
7.5.3.1 task_par	63
7.5.3.2 timespec_custom	63
7.5.4 Function Documentation	63
7.5.4.1 deadline_miss()	63
7.5.4.2 get_systime()	63
7.5.4.3 get_task_index()	63
7.5.4.4 ptask_init()	64
7.5.4.5 task_activate()	64
7.5.4.6 task_adline()	64
7.5.4.7 task_atime()	64
7.5.4.8 task_create()	65
7.5.4.9 task_deadline()	65
7.5.4.10 task_dmiss()	65
7.5.4.11 task_period()	65
7.5.4.12 task_set_deadline()	65
7.5.4.13 task_set_period()	66
7.5.4.14 time_add_ms()	66
7.5.4.15 time_cmp()	66
7.5.4.16 time_copy()	66
7.5.4.17 wait_for_activation()	66
7.5.4.18 wait_for_period()	67
7.5.4.19 wait_for_task_end()	67
7.5.5 Variable Documentation	67
7.5.5.1 ptask_policy	68
7.5.5.2 ptask_t0	68
7.5.5.3 tp	68
7.6 include/tasks.h File Reference	68
7.6.1 Detailed Description	69
7.6.2 Function Documentation	69
7.6.2.1 control_task()	70
7.6.2.2 display_task()	70
7.6.2.3 perception_task()	71
7.6.2.4 settings_task()	73
7.6.2.5 trajectory_task()	74
7.7 include/trajectory.h File Reference	75
7.7.1 Detailed Description	77
7.7.2 Function Documentation	77
7.7.2.1 trajectory_planning()	77

7.7.3 Variable Documentation	77
7.7.3.1 trajectory	77
7.7.3.2 trajectory_idx	78
7.8 include/utilities.h File Reference	78
7.8.1 Detailed Description	79
7.8.2 Function Documentation	79
7.8.2.1 angle_rotation_sprite()	79
7.8.2.2 init_cones()	80
7.8.2.3 load_cones_positions()	81
7.8.2.4 runtime()	81
7.9 include/vehicle.h File Reference	82
7.10 readme.md File Reference	82
7.11 src/control.c File Reference	82
7.11.1 Detailed Description	83
7.11.2 Function Documentation	84
7.11.2.1 autonomous_control()	84
7.11.2.2 keyboard_control()	85
7.11.3 Variable Documentation	86
7.11.3.1 pedal	86
7.11.3.2 steering	86
7.12 src/display.c File Reference	87
7.12.1 Detailed Description	88
7.12.2 Function Documentation	88
7.12.2.1 display_pedal()	89
7.12.2.2 draw_button()	89
7.12.2.3 draw_car()	89
7.12.2.4 draw_cone_map()	90
7.12.2.5 draw_controls()	91
7.12.2.6 draw_detected_cones()	92
7.12.2.7 draw_dir_arrow()	92
7.12.2.8 draw_lidar()	93
7.12.2.9 draw_perception()	93
7.12.2.10 draw_track()	94
7.12.2.11 draw_trajectory()	94
7.12.2.12 update_display()	95
7.12.3 Variable Documentation	96
7.12.3.1 btn_state_autonomous	96
7.12.3.2 btn_state_cones	96
7.12.3.3 btn_state_map	96
7.12.3.4 btn_state_perception	96
7.12.3.5 btn_state_traj	96
7.12.3.6 buttons	97

7.13 src/globals.c File Reference	98
7.13.1 Detailed Description	100
7.13.2 Variable Documentation	100
7.13.2.1 asphalt_gray	100
7.13.2.2 background	100
7.13.2.3 blue	100
7.13.2.4 car	100
7.13.2.5 car_angle	101
7.13.2.6 car_x	101
7.13.2.7 car_y	101
7.13.2.8 cone_radius	101
7.13.2.9 cones	101
7.13.2.10 control_panel	102
7.13.2.11 display_buffer	102
7.13.2.12 draw_mutex	102
7.13.2.13 grass_green	102
7.13.2.14 green	102
7.13.2.15 lidar_sem	103
7.13.2.16 measures	103
7.13.2.17 perception	103
7.13.2.18 pink	103
7.13.2.19 red	103
7.13.2.20 starting_cone_positions	103
7.13.2.21 steering_wheel	104
7.13.2.22 throttle_gauge	104
7.13.2.23 title	104
7.13.2.24 track	104
7.13.2.25 trajectory_bmp	104
7.13.2.26 white	105
7.13.2.27 X_MAX	105
7.13.2.28 Y_MAX	105
7.13.2.29 yellow	105
7.14 src/main.c File Reference	106
7.14.1 Detailed Description	107
7.14.2 Function Documentation	107
7.14.2.1 init_allegro()	107
7.14.2.2 init_bitmaps()	107
7.14.2.3 init_car()	108
7.14.2.4 init_control_panel()	108
7.14.2.5 init_perception()	109
7.14.2.6 init_track()	109
7.14.2.7 init_trajectory()	110

7.14.2.8 main()	110
7.14.3 Variable Documentation	111
7.14.3.1 car_bitmap_x	111
7.14.3.2 car_bitmap_y	111
7.14.3.3 car_x_px	111
7.14.3.4 car_y_px	111
7.14.3.5 filename	111
7.15 src/perception.c File Reference	113
7.15.1 Detailed Description	115
7.16 src/ptask.c File Reference	115
7.16.1 Macro Definition Documentation	116
7.16.1.1 PTASK_IMPLEMENTATION	116
7.17 src/tasks.c File Reference	116
7.17.1 Detailed Description	117
7.17.2 Function Documentation	117
7.17.2.1 control_task()	117
7.17.2.2 display_task()	118
7.17.2.3 is_mouse_over_button()	119
7.17.2.4 perception_task()	119
7.17.2.5 settings_task()	121
7.17.2.6 trajectory_task()	122
7.17.3 Variable Documentation	123
7.17.3.1 btn_state_autonomous	123
7.17.3.2 btn_state_cones	123
7.17.3.3 btn_state_map	124
7.17.3.4 btn_state_perception	124
7.17.3.5 btn_state_traj	124
7.17.3.6 buttons	124
7.18 src/trajectory.c File Reference	125
7.18.1 Detailed Description	126
7.18.2 Function Documentation	127
7.18.2.1 trajectory_planning()	127
7.18.3 Variable Documentation	128
7.18.3.1 trajectory	128
7.18.3.2 trajectory_idx	128
7.19 src/utilities.c File Reference	128
7.19.1 Detailed Description	129
7.19.2 Macro Definition Documentation	129
7.19.2.1 _POSIX_C_SOURCE	130
7.19.3 Function Documentation	132
7.19.3.1 angle_rotation_sprite()	132
7.19.3.2 init_cones()	132

7.19.3.3 load_cones_positions()	133
7.19.3.4 runtime()	134
7.19.4 Variable Documentation	134
7.19.4.1 offset_x	134
7.19.4.2 offset_y	134
7.20 src/vehicle.c File Reference	135
7.20.1 Macro Definition Documentation	135
7.20.1.1 NON_CINEMATIC_MODEL	135

Chapter 1

2D FSAE Simulation

1.1 Project Overview

This project is a 2D simulation of a Formula SAE (FSAE) vehicle, designed to model vehicle dynamics, perception, trajectory planning, and control systems. It provides a platform for testing and visualizing autonomous driving algorithms in a simplified environment.

1.2 Key Features

- **Vehicle Dynamics:** Simulates basic vehicle movement based on pedal and steering inputs.
- **LiDAR-based Perception:** Implements cone detection using simulated LiDAR measurements and a circle Hough transform.
- **Trajectory Planning:** Generates a driving trajectory by connecting detected cones.
- **Autonomous Control:** Includes an autonomous control system to follow the planned trajectory.
- **Manual Control:** Allows keyboard control of the vehicle.
- **Visualization:** Uses the Allegro library to provide a 2D graphical representation of the simulation, including the car, track, cones, and planned trajectory.
- **Periodic Tasks:** Utilizes a periodic task scheduler for managing different simulation components.

1.3 Project Structure

The project is organized into several modules:

- `**src/main.c:**` Initializes the simulation, creates tasks, and manages the main loop.
- `**src/vehicle.c:**` Implements the vehicle dynamics model.
- `**src/perception.c:**` Handles LiDAR data processing and cone detection.
- `**src/trajectory.c:**` Plans the driving trajectory based on detected cones.
- `**src/control.c:**` Implements control algorithms for both manual and autonomous driving.

- `**src/display.c:**` Manages the graphical display of the simulation.
- `**src/tasks.c:**` Defines the periodic tasks for perception, trajectory planning, control, and display.
- `**src/utilities.c:**` Provides utility functions for cone management and runtime measurement.
- `**src/globals.c:**` Defines global variables and constants used throughout the simulation.
- `**src/ptask.c:**` Implements the periodic task scheduler.
- `**include/*.h:**` Header files declaring the interfaces for each module.

1.4 Global Call Graph

1.5 Task Descriptions

- **Perception Task:** Acquires LiDAR measurements, detects cones, and updates the map.
- **Trajectory Task:** Plans the trajectory based on the detected cones.
- **Control Task:** Controls the vehicle either manually or autonomously.
- **Display Task:** Updates the graphical display of the simulation.

1.6 Building and Running the Simulation

1) Ensure you have Allegro 4 installed *(On Ubuntu/Debian:* `sudo apt-get install liballegro4.2 liballegro4.2-dev*`)*.

2) Other dependencies for this project:

- *libyaml*: `git clone https://github.com/yaml/libyaml.git`

3) Use the provided Makefile to build the project.

4) Run the executable `make`.

Chapter 2

Module Index

2.1 Modules

Here is a list of all modules:

Cone Detection and Mapping	9
Cone Detection and Mapping	14
Vehicle model and control	14

Chapter 3

Class Index

3.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

Button	17
candidate_cone Represents a candidate cone that might be validated after multiple detections	18
cone Represents a cone detected in the environment	20
cone_border Represents a border of a cone as seen in a LiDAR scan	21
Hough_circle_point_t Represents a point in the Hough circle transformation space	22
pointcloud_t Contains the measures of distance made by the LiDAR at each angle	23
task_par	24
timespec_custom	26
waypoint Plans the trajectory for the car based on its current state and detected obstacles	27

Chapter 4

File Index

4.1 File List

Here is a list of all files with brief descriptions:

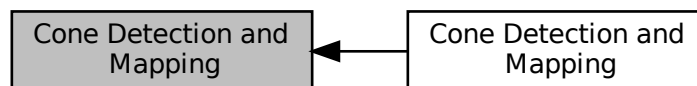
include/control.h	Declarations for vehicle control functions and variables in the 2D simulator	29
include/display.h	Header file declaring display rendering functions	34
include/globals.h	Global constants, variables, and structures for the simulation and visualization system	43
include/perception.h	Header file for perception-related types, constants, and function declarations	55
include/ptask.h	Interface for managing periodic tasks and time utilities	58
include/tasks.h	Declaration of task thread functions for the simulation	68
include/trajectory.h	Provides declarations for trajectory planning based on cone detection	75
include/utilities.h	Utility function prototypes for cone management and runtime tasks	78
include/vehicle.h		82
src/control.c	Implements vehicle control functions including both manual (keyboard) and autonomous (center-line based) control	82
src/display.c	Rendering and display update functions for the 2D simulation using Allegro	87
src/globals.c	Global definitions and state variables for the 2D FSAE simulation	98
src/main.c	Main implementation of the 2D FSAE Simulation	106
src/perception.c	Implements LiDAR-based cone detection and mapping using circle Hough transform	113
src/ptask.c		115
src/tasks.c	Contains periodic task functions for LiDAR perception, trajectory planning, vehicle control, and display update in a 2D simulation environment	116
src/trajectory.c	Implements trajectory planning based on detected cones	125
src/utilities.c	Utility functions for cone initialization, YAML cone loading, angle rotation conversion, and runtime performance measurement	128
src/vehicle.c		135

Chapter 5

Module Documentation

5.1 Cone Detection and Mapping

Collaboration diagram for Cone Detection and Mapping:



Modules

- [Cone Detection and Mapping](#)

Files

- file [perception.c](#)

Implements LiDAR-based cone detection and mapping using circle Hough transform.

Functions

- void [check_nearest_point](#) (int angle, float new_point_x, float new_point_y, int color, [cone_border](#) *cone_↔ borders)
- void [lidar](#) ([pointcloud_t](#) *measures)
- void [init_cone_borders](#) ([cone_border](#) *cone_borders)
- void [calculate_circle_points](#) (float center_x, float center_y, int color, [cone](#) *circle_points)
- void [find_closest_points](#) ([Hough_circle_point_t](#) *circumference_points, float point_x, float point_↔ y, [Hough_circle_point_t](#) *reference_points, int ref_size)
- void [find_local_minima](#) ([Hough_circle_point_t](#) *points, int *first_min, int *second_min)
- float * [find_cone_center](#) ([Hough_circle_point_t](#) *possible_centers, int center_count)
- void [mapping](#) ([cone](#) *detected_cones)
- void [update_map](#) ([cone](#) *detected_cones)

Variables

- const int `sliding_window` = 360
- const int `angle_step` = 1
- int `start_angle` = 0
- const float `ignore_distance` = 0.5f
- const float `distance_resolution` = 0.01f
- `cone_detected_cones` [MAX_DETECTED_CONES]
- int `n_candidates` = 0
- `candidate_cone_candidates` [MAX_CANDIDATES]
- int `track_map_idx` = 0
- `cone_track_map` [MAX_CONES_MAP]

5.1.1 Detailed Description

5.1.2 Function Documentation

5.1.2.1 `calculate_circle_points()`

```
void calculate_circle_points (
    float center_x,
    float center_y,
    int color,
    cone * circle_points )
```

5.1.2.2 `check_nearest_point()`

```
void check_nearest_point (
    int angle,
    float new_point_x,
    float new_point_y,
    int color,
    cone_border * cone_borders )
```

Here is the caller graph for this function:



5.1.2.3 find_closest_points()

```
void find_closest_points (
    Hough_circle_point_t * circumference_points,
    float point_x,
    float point_y,
    Hough_circle_point_t * reference_points,
    int ref_size )
```

5.1.2.4 find_cone_center()

```
float* find_cone_center (
    Hough_circle_point_t * possible_centers,
    int center_count )
```

5.1.2.5 find_local_minima()

```
void find_local_minima (
    Hough_circle_point_t * points,
    int * first_min,
    int * second_min )
```

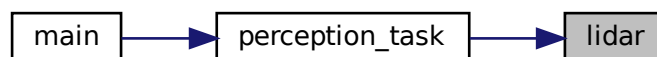
5.1.2.6 init_cone_borders()

```
void init_cone_borders (
    cone_border * cone_borders )
```

5.1.2.7 lidar()

```
void lidar (
    pointcloud_t * measures )
```

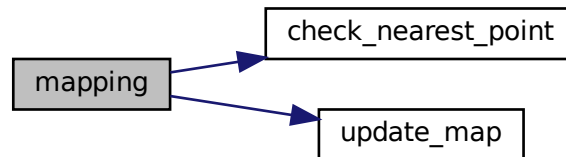
Here is the caller graph for this function:



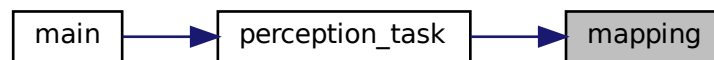
5.1.2.8 mapping()

```
void mapping (  
    cone * detected_cones )
```

Here is the call graph for this function:



Here is the caller graph for this function:



5.1.2.9 update_map()

```
void update_map (  
    cone * detected_cones )
```

Here is the caller graph for this function:



5.1.3 Variable Documentation

5.1.3.1 angle_step

```
const int angle_step = 1
```

5.1.3.2 candidates

```
candidate_cone candidates[MAX_CANDIDATES]
```

5.1.3.3 detected_cones

```
cone detected_cones[MAX_DETECTED_CONES]
```

5.1.3.4 distance_resolution

```
const float distance_resolution = 0.01f
```

5.1.3.5 ignore_distance

```
const float ignore_distance = 0.5f
```

5.1.3.6 n_candidates

```
int n_candidates = 0
```

5.1.3.7 sliding_window

```
const int sliding_window = 360
```

5.1.3.8 start_angle

```
int start_angle = 0
```

5.1.3.9 track_map

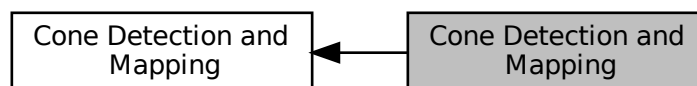
```
cone track_map[MAX_CONES_MAP]
```

5.1.3.10 track_map_idx

```
int track_map_idx = 0
```

5.2 Cone Detection and Mapping

Collaboration diagram for Cone Detection and Mapping:



5.3 Vehicle model and control

Macros

- `#define M_PI 3.14159265358979323846`
Updates the vehicle's position and orientation based on pedal and steering inputs.

Functions

- void `vehicle_model` (float `pedal`, float `target_steering`)
Simulates the vehicle's movement by updating its position and orientation.
- void `check_collisions` ()

5.3.1 Detailed Description

5.3.2 Macro Definition Documentation

5.3.2.1 M_PI

```
#define M_PI 3.14159265358979323846
```

Updates the vehicle's position and orientation based on pedal and steering inputs.

This function simulates the movement of a vehicle using a simple bicycle model (Ackermann steering). It updates the current position (`car_x`, `car_y`) and the heading (`car_angle`) of the vehicle according to the pedal input (acceleration or braking) and the steering angle over a fixed time step.

The simulation parameters include:

- A time step (`dt`) for the simulation.
- A mass of the vehicle, which is used to compute acceleration from the force.
- A fixed wheelbase, representing the distance between the front and rear axles.
- A maximum achievable speed and a maximum braking force.

When the pedal input is positive, the vehicle accelerates towards a speed that is a proportion of `maxSpeed`. If the pedal input is not positive, the vehicle is assumed to be braking, and the deceleration is computed using the current speed and a maximum braking factor.

The vehicle's position is updated based on its current speed and orientation. The steering angle influences the change in the vehicle's heading only if the vehicle is moving above a small threshold speed. The angle conversion between degrees and radians is used to interface with the trigonometric functions.

Parameters

in, out	<code>car_x</code>	Pointer to the vehicle's x-coordinate. Updated based on the vehicle's motion.
in, out	<code>car_y</code>	Pointer to the vehicle's y-coordinate. Updated based on the vehicle's motion.
in, out	<code>car_angle</code>	Pointer to the vehicle's heading in degrees. Updated based on the computed orientation.
in	<code>pedal</code>	The pedal input; a positive value corresponds to acceleration while a non-positive value initiates braking.
in	<code>steering</code>	The steering input that determines the rate of change of the vehicle's heading.

5.3.3 Function Documentation

5.3.3.1 check_collisions()

```
void check_collisions ( )
```

Here is the caller graph for this function:



5.3.3.2 vehicle_model()

```

void vehicle_model (
    float pedal,
    float steering )
  
```

Simulates the vehicle's movement by updating its position and orientation.

This function computes and updates the vehicle's state based on the input controls. The vehicle's x and y coordinates, as well as its orientation angle, are modified to reflect the effect of the applied pedal (acceleration/brake) and steering inputs.

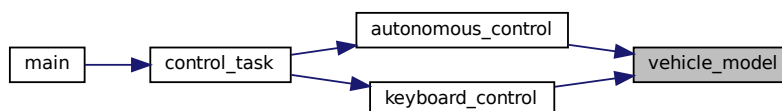
Parameters

<i>car_x</i>	Pointer to the vehicle's x-coordinate. The updated x position is stored here.
<i>car_y</i>	Pointer to the vehicle's y-coordinate. The updated y position is stored here.
<i>car_angle</i>	Pointer to the vehicle's current orientation angle (in degrees or radians). The updated angle is stored here.
<i>pedal</i>	Control input for acceleration (or braking). A non-zero value indicates a change in speed.
<i>steering</i>	Control input for the steering direction. A non-zero value modifies the vehicle's heading.

Note

Ensure that the pointers provided (*car_x*, *car_y*, *car_angle*) are valid and point to allocated memory. The scaling and meaning of the 'pedal' and 'steering' inputs should correspond to the mechanics of the vehicle model being simulated.

Here is the caller graph for this function:



Chapter 6

Class Documentation

6.1 Button Struct Reference

```
#include <display.h>
```

Public Attributes

- int `x`
- int `y`
- int `width`
- int `height`
- const char * `text`
- int * `value`

6.1.1 Member Data Documentation

6.1.1.1 height

```
int Button::height
```

6.1.1.2 text

```
const char* Button::text
```

6.1.1.3 value

```
int* Button::value
```

6.1.1.4 width

```
int Button::width
```

6.1.1.5 x

```
int Button::x
```

6.1.1.6 y

```
int Button::y
```

The documentation for this struct was generated from the following file:

- include/[display.h](#)

6.2 candidate_cone Struct Reference

Represents a candidate cone that might be validated after multiple detections.

```
#include <perception.h>
```

Public Attributes

- float [x](#)
- float [y](#)
- int [color](#)
- int [detections](#)

6.2.1 Detailed Description

Represents a candidate cone that might be validated after multiple detections.

This structure is used for maintaining a candidate cone before it is confirmed. It stores the position, color, and the number of detections that have contributed to this candidate.

Members:

- x: X position of the candidate cone.
- y: Y position of the candidate cone.
- color: Color of the candidate cone.
- detections: Count of how many times this candidate has been detected.

6.2.2 Member Data Documentation

6.2.2.1 color

```
int candidate_cone::color
```

6.2.2.2 detections

```
int candidate_cone::detections
```

6.2.2.3 x

```
float candidate_cone::x
```

6.2.2.4 y

```
float candidate_cone::y
```

The documentation for this struct was generated from the following file:

- [include/perception.h](#)

6.3 cone Struct Reference

Represents a cone detected in the environment.

```
#include <globals.h>
```

Public Attributes

- float [x](#)
- float [y](#)
- int [color](#)

6.3.1 Detailed Description

Represents a cone detected in the environment.

This structure stores the position and color information for a cone. The positions are stored in floating point values in meters and can be converted to pixels when rendering. The color is represented using the Allegro color format.

Members:

- x: X position of the cone.
- y: Y position of the cone.
- color: Color of the cone (using Allegro color format).

6.3.2 Member Data Documentation

6.3.2.1 color

```
int cone::color
```

Color (in Allegro color format)

6.3.2.2 x

```
float cone::x
```

X position of the cone (in meters, converted to px when drawn)

6.3.2.3 y

```
float cone::y
```

Y position of the cone

The documentation for this struct was generated from the following file:

- [include/globals.h](#)

6.4 cone_border Struct Reference

Represents a border of a cone as seen in a LiDAR scan.

```
#include <perception.h>
```

Public Attributes

- int [angles](#) [[MAX_POINTS_PER_CONE](#)]
- int [color](#)

6.4.1 Detailed Description

Represents a border of a cone as seen in a LiDAR scan.

This structure collects indices from the LiDAR scan that correspond to the boundaries of a cone. It also stores the color information associated with that cone border.

Members:

- [angles](#): An array of indices (angles) in the LiDAR scan corresponding to the same cone border.
- [color](#): Color of the cone border.

6.4.2 Member Data Documentation

6.4.2.1 angles

```
int cone_border::angles[MAX\_POINTS\_PER\_CONE]
```

Indices (angles) in the LiDAR scan that map to the same cone border

6.4.2.2 color

```
int cone_border::color
```

Color of the cone

The documentation for this struct was generated from the following file:

- [include/perception.h](#)

6.5 Hough_circle_point_t Struct Reference

Represents a point in the Hough circle transformation space.

```
#include <perception.h>
```

Public Attributes

- float [x](#)
- float [y](#)
- float [distance](#)
- int [color](#)

6.5.1 Detailed Description

Represents a point in the Hough circle transformation space.

This structure is used to store information about a point in the Hough circle transformation space, including its position, distance, and color.

6.5.2 Member Data Documentation

6.5.2.1 color

```
int Hough_circle_point_t::color
```

6.5.2.2 distance

```
float Hough_circle_point_t::distance
```

6.5.2.3 x

```
float Hough_circle_point_t::x
```

6.5.2.4 y

```
float Hough_circle_point_t::y
```

The documentation for this struct was generated from the following file:

- [include/perception.h](#)

6.6 pointcloud_t Struct Reference

Contains the measures of distance made by the LiDAR at each angle.

```
#include <globals.h>
```

Public Attributes

- float [point_x](#)
- float [point_y](#)
- float [distance](#)
- int [color](#)

6.6.1 Detailed Description

Contains the measures of distance made by the LiDAR at each angle.

6.6.2 Member Data Documentation

6.6.2.1 color

```
int pointcloud_t::color
```

6.6.2.2 distance

```
float pointcloud_t::distance
```

6.6.2.3 point_x

```
float pointcloud_t::point_x
```

6.6.2.4 point_y

```
float pointcloud_t::point_y
```

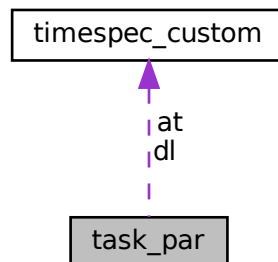
The documentation for this struct was generated from the following file:

- [include/globals.h](#)

6.7 task_par Struct Reference

```
#include <ptask.h>
```

Collaboration diagram for task_par:



Public Attributes

- [int arg](#)
- [int period](#)
- [int deadline](#)
- [int prio](#)
- [int dmiss](#)
- [timespec_custom at](#)
- [timespec_custom dl](#)
- [pthread_t tid](#)
- [sem_t asem](#)

6.7.1 Member Data Documentation

6.7.1.1 arg

```
int task_par::arg
```

6.7.1.2 asem

```
sem_t task_par::asem
```

6.7.1.3 at

```
timespec_custom task_par::at
```

6.7.1.4 deadline

```
int task_par::deadline
```

6.7.1.5 dl

```
timespec_custom task_par::dl
```

6.7.1.6 dmiss

```
int task_par::dmiss
```

6.7.1.7 period

```
int task_par::period
```

6.7.1.8 prio

```
int task_par::prio
```

6.7.1.9 tid

```
pthread_t task_par::tid
```

The documentation for this struct was generated from the following file:

- [include/ptask.h](#)

6.8 timespec_custom Struct Reference

```
#include <ptask.h>
```

Public Attributes

- time_t [tv_sec](#)
- long [tv_nsec](#)

6.8.1 Member Data Documentation

6.8.1.1 tv_nsec

```
long timespec_custom::tv_nsec
```

6.8.1.2 tv_sec

```
time_t timespec_custom::tv_sec
```

The documentation for this struct was generated from the following file:

- [include/ptask.h](#)

6.9 waypoint Struct Reference

Plans the trajectory for the car based on its current state and detected obstacles.

```
#include <trajectory.h>
```

Public Attributes

- float [x](#)
- float [y](#)

6.9.1 Detailed Description

Plans the trajectory for the car based on its current state and detected obstacles.

Represents a point in 2D space.

This function calculates the trajectory by considering the car's position and orientation, as well as the positions of detected cones. The resultant waypoints are stored in the provided trajectory array.

Parameters

<i>car_x</i>	The x-coordinate of the car's current position.
<i>car_y</i>	The y-coordinate of the car's current position.
<i>car_angle</i>	The heading angle of the car in radians.
<i>trajectory</i>	Pointer to an array of waypoint structures where the planned trajectory will be stored.

This structure is used to define a waypoint with x and y coordinates.

6.9.2 Member Data Documentation

6.9.2.1 [x](#)

```
float waypoint::x
```

6.9.2.2 [y](#)

```
float waypoint::y
```

The documentation for this struct was generated from the following file:

- include/[trajectory.h](#)

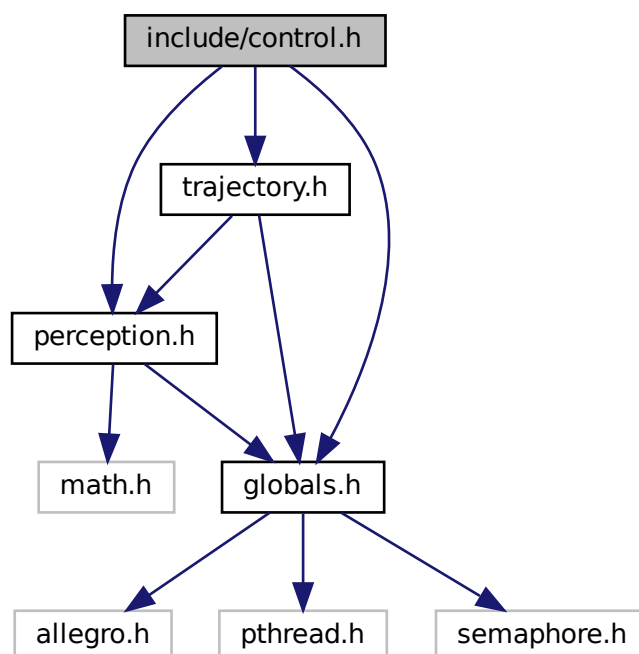
Chapter 7

File Documentation

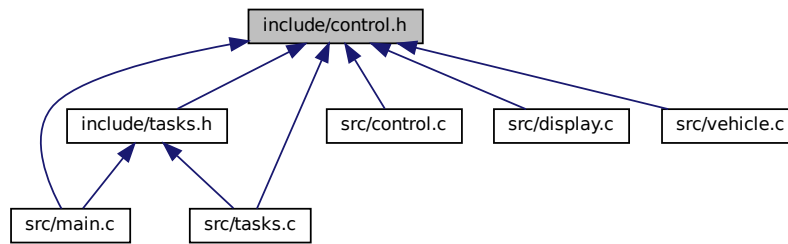
7.1 include/control.h File Reference

Declarations for vehicle control functions and variables in the 2D simulator.

```
#include "trajectory.h"  
#include "globals.h"  
#include "perception.h"  
Include dependency graph for control.h:
```



This graph shows which files directly or indirectly include this file:



Functions

- void [keyboard_control](#) ()
Adjusts vehicle controls based on keyboard input.
- void [autonomous_control](#) ([waypoint](#) *trajectory)
Autonomous control routine using centerline waypoints.

Variables

- float [steering](#)
Controls the vehicle using keyboard input.
- float [pedal](#)
- [waypoint reordered_ahead](#) [MAX_WAYPOINTS]
- int [wp_ahead_idx](#)

7.1.1 Detailed Description

Declarations for vehicle control functions and variables in the 2D simulator.

This header file is part of the 2D simulation project and provides the interface for controlling the simulated vehicle. It supports both manual (keyboard-driven) and autonomous (trajectory-based) modes.

Global Variables:

- [steering](#): Current steering angle in radians.
- [pedal](#): Throttle input as a normalized value in the range [0, 1].

Dependencies:

- [trajectory.h](#): Used for waypoint and trajectory definitions.
- [globals.h](#): Contains global variables and settings.
- [perception.h](#): Provides functionalities related to situational awareness.

7.1.2 Function Documentation

7.1.2.1 autonomous_control()

```
void autonomous_control (
    waypoint * center_waypoints )
```

Autonomous control routine using centerline waypoints.

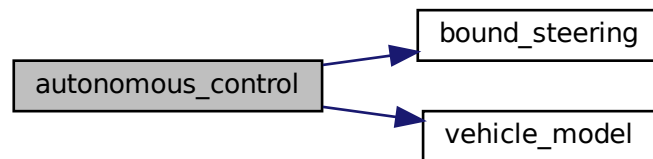
Implements an autonomous control strategy based on a provided centerline represented by an array of waypoints. The function follows these steps:

- Counts the number of valid centerline waypoints (terminated when a waypoint with $x < 0.0f$ is found).
- Filters the centerline to extract waypoints that are located ahead of the vehicle using the `is_in_front` helper.
- Identifies the closest valid waypoint ahead, and depending on the availability of neighboring points, computes a reference trajectory vector. This is done in one of three ways:
 - If at least three valid ahead waypoints exist, the reference vector is computed from the previous to the next waypoint surrounding the closest ahead waypoint.
 - If there are only one or two ahead points (but at least two total waypoints), the reference vector is derived by combining the vector from the vehicle to the last waypoint and the segment between the last two waypoints.
 - If only one waypoint is available, the vector from the vehicle to that waypoint is used as the reference.
- The computed reference trajectory is normalized. If normalization fails, a default forward direction is used.
- The vehicle's current heading is computed as a unit vector based on `local_car_angle`.
- The required steering correction (delta) is obtained by computing the sine of the angle difference through the 2D cross product between the normalized reference vector and the heading vector.
- A constant pedal value is applied.
- Finally, the updated control signals (pedal and delta for steering) are applied to the vehicle by calling `vehicle->_model`.

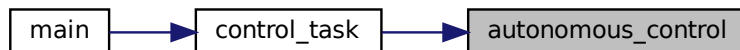
Parameters

in	<i>center_waypoints</i>	Pointer to an array of waypoints representing the desired centerline trajectory. The array should be terminated by a waypoint with $x < 0.0f$.
----	-------------------------	---

Here is the call graph for this function:



Here is the caller graph for this function:



7.1.2.2 keyboard_control()

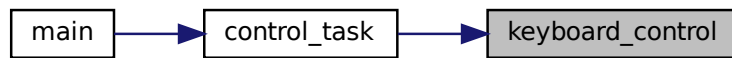
```
void keyboard_control ( )
```

Adjusts vehicle controls based on keyboard input.

This function processes the state of keyboard keys to adjust the vehicle's pedal (speed) and steering angle. It increments or decrements the pedal value and steering angle within pre-defined limits depending on the keys pressed (e.g., `KEY_UP`, `KEY_DOWN`, `KEY_LEFT`, `KEY_RIGHT`). After updating these control signals, the vehicle's state is updated by calling the `vehicle_model` function. Here is the call graph for this function:



Here is the caller graph for this function:



7.1.3 Variable Documentation

7.1.3.1 pedal

```
float pedal [extern]
```

7.1.3.2 reordered_ahead

```
waypoint reordered_ahead[MAX_WAYPOINTS] [extern]
```

7.1.3.3 steering

```
float steering [extern]
```

Controls the vehicle using keyboard input.

Processes user keyboard inputs to modify the vehicle's position and orientation. The function updates the car's x and y coordinates, as well as its angular orientation (in degrees).

Parameters

<i>car_x</i>	Pointer to the car's x-coordinate.
<i>car_y</i>	Pointer to the car's y-coordinate.
<i>car_angle</i>	Pointer to the car's current angle (in degrees).

Controls the vehicle autonomously along a given trajectory.

Uses the provided trajectory (a sequence of waypoints) to automatically adjust the vehicle's steering and pedal inputs. This function updates the car's position and orientation by computing the necessary control actions based on the navigation path.

Parameters

<i>car_x</i>	Pointer to the car's x-coordinate.
<i>car_y</i>	Pointer to the car's y-coordinate.
<i>car_angle</i>	Pointer to the car's current orientation angle (in degrees).
<i>trajectory</i>	Pointer to the waypoint structure that defines the trajectory.

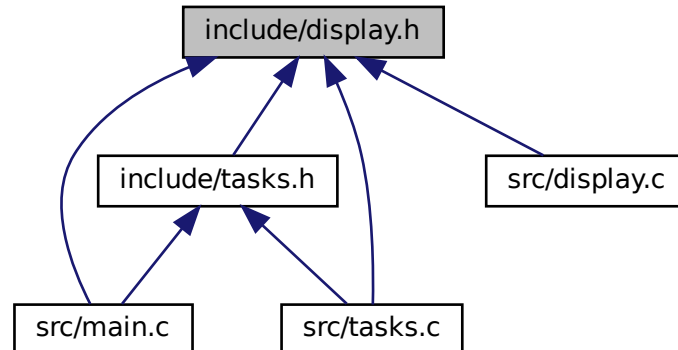
7.1.3.4 wp_ahead_idx

```
int wp_ahead_idx [extern]
```

7.2 include/display.h File Reference

Header file declaring display rendering functions.

This graph shows which files directly or indirectly include this file:

**Classes**

- struct [Button](#)

Macros

- `#define` [num_buttons](#) 5

Functions

- void `draw_dir_arrow` ()
Draws a directional arrow.
- void `draw_car` (float `car_x`, float `car_y`, int `car_angle`)
Draws a car shape on the display.
- void `draw_track` ()
Draws the track.
- void `draw_lidar` (`pointcloud_t` *`measures`)
Draws lidar measurements.
- void `draw_detected_cones` (`cone` *`detected_cones`)
Draws detected cones.
- void `draw_cone_map` (`cone` *`track_map`, int `track_map_idx`)
Draws the cone map (track map).
- void `draw_perception` ()
Draws perception results.
- void `draw_trajectory` (`waypoint` *`trajectory`)
Draws the planned trajectory.
- void `update_display` ()
Updates the display.

Variables

- int `btn_state_cones`
- int `btn_state_perception`
- int `btn_state_map`
- int `btn_state_traj`
- int `btn_state_autonomous`

7.2.1 Detailed Description

Header file declaring display rendering functions.

This file contains declarations for functions that handle the drawing of various display elements including car representation, lidar measurements, track, perception, trajectory, and more. These functions provide an interface for visualizing simulation data.

Note

This header relies on types and globals defined in "globals.h".

7.2.2 Macro Definition Documentation

7.2.2.1 num_buttons

```
#define num_buttons 5
```

7.2.3 Function Documentation

7.2.3.1 draw_car()

```
void draw_car (
    float car_x,
    float car_y,
    int car_angle )
```

Draws a car shape on the display.

Parameters

<i>car_x</i>	X-coordinate of the car's position.
<i>car_y</i>	Y-coordinate of the car's position.
<i>car_angle</i>	Orientation angle of the car (typically in degrees).

Renders a graphical representation of the car using its position and orientation.

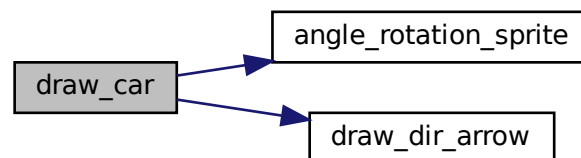
Draws a car shape on the display.

Calculates the proper position for the car sprite based on its coordinates in the world space (scaled by pixels per meter) and draws it using Allegro's `rotate_scaled_sprite` function. In `DEBUG` mode, the function also draws view angle lines to indicate the car's field of view.

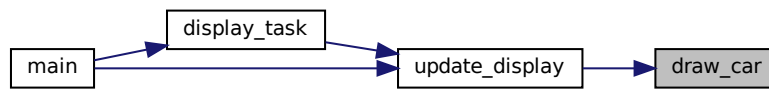
Parameters

<i>car_x</i>	The car's x-coordinate in world units.
<i>car_y</i>	The car's y-coordinate in world units.
<i>car_angle</i>	The car's angle in degrees.

Here is the call graph for this function:



Here is the caller graph for this function:



7.2.3.2 draw_cone_map()

```

void draw_cone_map (
    cone * track_map,
    int track_map_idx )
  
```

Draws the cone map (track map).

Parameters

<i>track_map</i>	Pointer to an array of cone structures representing the map.
<i>track_map_idx</i>	Index representing the current position or number of cones on the track map.

This function displays the cone-based track map, helping the user to visualize the layout and key points along the track.

Draws the cone map (track map).

Iterates through the track map data and draws a white filled circle at each cone's position, properly mapping simulation world coordinates to the perception window.

Parameters

<i>track_map</i>	Pointer to an array of cones representing the track map.
<i>track_map_idx</i>	The number of valid entries in the track map array.

Here is the caller graph for this function:



7.2.3.3 draw_detected_cones()

```
void draw_detected_cones (
    cone * detected_cones )
```

Draws detected cones.

Parameters

<i>detected_cones</i>	Pointer to a cone structure that holds information about detected cones.
-----------------------	--

Renders the detected cones on the display, which could represent position markers or obstacles in the simulation environment.

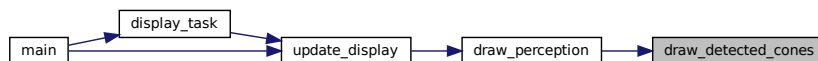
Draws detected cones.

Iterates through the array of detected cones and draws a filled circle at each cone's location, after mapping the coordinates from the simulation world to the perception window.

Parameters

<i>detected_cones</i>	Pointer to an array of detected cone structures.
-----------------------	--

Here is the caller graph for this function:



7.2.3.4 draw_dir_arrow()

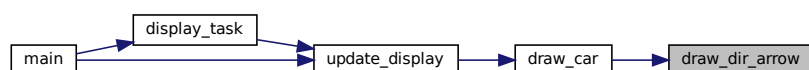
```
void draw_dir_arrow ( )
```

Draws a directional arrow.

This function is used to render an arrow representing direction on the display. The arrow can be used as an indicator for direction in the simulation.

Draws a directional arrow.

Computes the arrow's start (at the car's position) and end points based on the car angle, using cosine and sine functions. The arrow shaft and head are drawn as thick green lines, with the head drawn at two angles to form a head shape. Here is the caller graph for this function:



7.2.3.5 draw_lidar()

```
void draw_lidar (
    pointcloud_t * measures )
```

Draws lidar measurements.

Parameters

<i>measures</i>	Pointer to a pointcloud_t structure containing lidar data.
-----------------	--

Processes and graphically displays the lidar data as a set of points or a cloud, allowing visualization of the detected obstacles or environment.

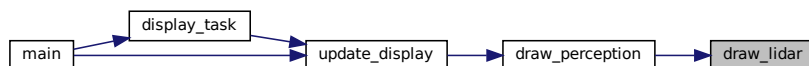
Draws lidar measurements.

Clears the perception bitmap and centers it on the car's position. For each lidar measurement, computes the global-to-perception window mapping for the detected point and draws a line from the car's center to that point. The color of the line depends on whether a cone was detected or not.

Parameters

<i>measures</i>	Pointer to an array of lidar measurements.
-----------------	--

Here is the caller graph for this function:



7.2.3.6 draw_perception()

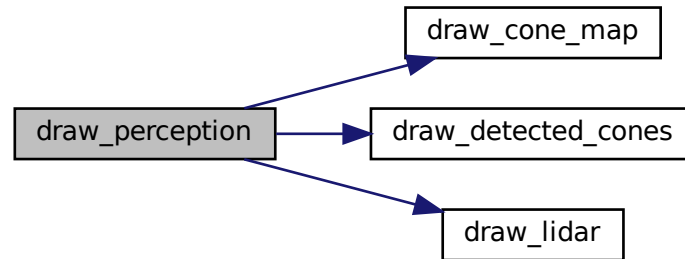
```
void draw_perception ( )
```

Draws perception results.

This function is responsible for visualizing the outcomes of the perception process, which might include sensor fusion results or object detection overlays.

Draws perception results.

Combines the lidar lines, detected cones, and cone map visualizations to create the perception overlay. Afterwards, the perception bitmap is blended onto the main display buffer offset in relation to the car's position. Here is the call graph for this function:



Here is the caller graph for this function:



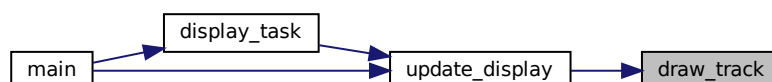
7.2.3.7 draw_track()

```
void draw_track ( )
```

Draws the track.

This function draws the track layout on the display, which might include boundaries, lanes, or other information necessary for simulation or visualization.

Checks that the track bitmap has valid dimensions, and if so, draws the track image onto the display buffer. If the bitmap dimensions are invalid, an error message is shown. Here is the caller graph for this function:



7.2.3.8 draw_trajectory()

```
void draw_trajectory (
    waypoint * trajectory )
```

Draws the planned trajectory.

Parameters

<i>trajectory</i>	Pointer to an array of waypoint structures representing the planned path.
-------------------	---

Visualizes the computed trajectory on the display, providing insight into planned routes or maneuvers.

Draws the planned trajectory.

Clears and sets the background for the trajectory bitmap and iterates over each trajectory waypoint, drawing each as a filled circle. In DEBUG mode, the waypoint index is also displayed and an extra line is drawn from the car sprite center.

Parameters

<i>trajectory</i>	Pointer to an array of waypoint structures representing the car's trajectory.
-------------------	---

Here is the caller graph for this function:



7.2.3.9 update_display()

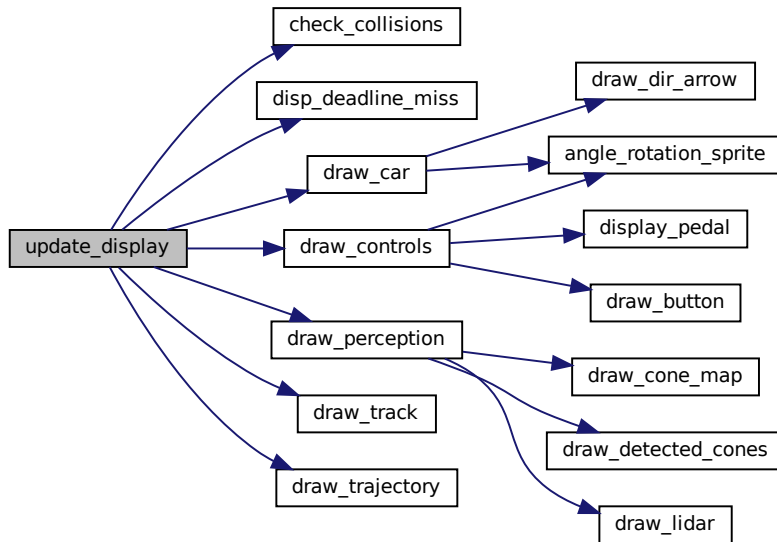
```
void update_display ( )
```

Updates the display.

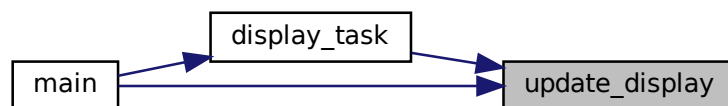
This function refreshes the display, ensuring that all drawn elements are updated according to the latest simulation data.

Updates the display.

Locks the drawing mutex and sequentially renders the background, track, car, perception, trajectory, title text, and controls on the display buffer. Once all elements are rendered, the display buffer is blitted to the screen, and the mutex is subsequently unlocked. Here is the call graph for this function:



Here is the caller graph for this function:



7.2.4 Variable Documentation

7.2.4.1 btn_state_autonomous

```
int btn_state_autonomous [extern]
```

7.2.4.2 btn_state_cones

```
int btn_state_cones [extern]
```

7.2.4.3 btn_state_map

```
int btn_state_map [extern]
```

7.2.4.4 btn_state_perception

```
int btn_state_perception [extern]
```

7.2.4.5 btn_state_traj

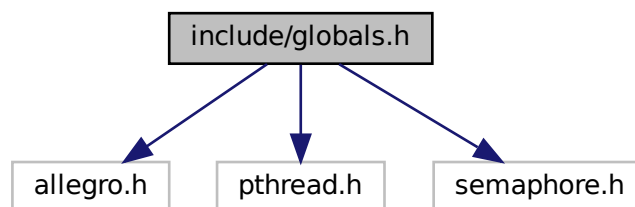
```
int btn_state_traj [extern]
```

7.3 include/globals.h File Reference

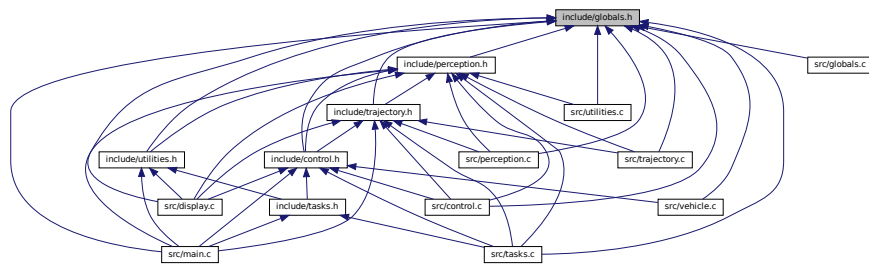
Global constants, variables, and structures for the simulation and visualization system.

```
#include <allegro.h>
#include <pthread.h>
#include <semaphore.h>
```

Include dependency graph for globals.h:



This graph shows which files directly or indirectly include this file:



Classes

- struct [cone](#)
Represents a cone detected in the environment.
- struct [pointcloud_t](#)
Contains the measures of distance made by the LiDAR at each angle.

Macros

- #define [PROFILING](#)
- #define [PERCEPTION_PERIOD](#) 50
- #define [TRAJECTORY_PERIOD](#) 100
- #define [CONTROL_PERIOD](#) 50
- #define [DISPLAY_PERIOD](#) 17
- #define [SETTINGS_PERIOD](#) 100
- #define [PERCEPTION_DEADLINE](#) [PERCEPTION_PERIOD](#)
- #define [TRAJECTORY_DEADLINE](#) [TRAJECTORY_PERIOD](#)
- #define [CONTROL_DEADLINE](#) [CONTROL_PERIOD](#)
- #define [DISPLAY_DEADLINE](#) [DISPLAY_PERIOD](#)
- #define [SETTINGS_DEADLINE](#) [SETTINGS_PERIOD](#)
- #define [PERCEPTION_PRIORITY](#) 15
- #define [TRAJECTORY_PRIORITY](#) 20
- #define [CONTROL_PRIORITY](#) 5
- #define [DISPLAY_PRIORITY](#) 30
- #define [SETTINGS_PRIORITY](#) 35
- #define [PERCEPTION_ID](#) 1
- #define [TRAJECTORY_ID](#) 2
- #define [CONTROL_ID](#) 3
- #define [DISPLAY_ID](#) 4
- #define [SETTINGS_ID](#) 5
- #define [px_per_meter](#) 100
- #define [deg2rad](#) 0.017453292519943295769236907684886f
- #define [maxThrottleHeight](#) 100
- #define [MAX_CONES_MAP](#) 3000
- #define [MAX_DETECTED_CONES](#) 360

Variables

- int [dl_miss_perception](#)
- int [dl_miss_trajectory](#)
- int [dl_miss_control](#)
- int [dl_miss_display](#)
- int [dl_miss_settings](#)
- pthread_mutex_t [draw_mutex](#)
- pthread_mutex_t [map_mutex](#)
- pthread_mutex_t [car_mutex](#)
- const char * [title](#)
- const int [X_MAX](#)
- const int [Y_MAX](#)
- BITMAP * [control_panel](#)
- BITMAP * [steering_wheel](#)
- BITMAP * [throttle_gauge](#)
- BITMAP * [background](#)
- BITMAP * [track](#)
- BITMAP * [car](#)
- BITMAP * [perception](#)
- BITMAP * [trajectory_bmp](#)
- BITMAP * [display_buffer](#)
- int [grass_green](#)
- int [asphalt_gray](#)
- int [white](#)
- int [pink](#)
- int [yellow](#)
- int [blue](#)
- int [red](#)
- int [green](#)
- float [car_x](#)
- float [car_y](#)
- int [car_angle](#)
- const float [cone_radius](#)
- [cone](#) [cones](#) [[MAX_CONES_MAP](#)]
- [cone](#) [starting_cone_positions](#) [[MAX_CONES_MAP](#)]
- [pointcloud_t](#) [measures](#) [[MAX_DETECTED_CONES](#)]
- sem_t [lidar_sem](#)

7.3.1 Detailed Description

Global constants, variables, and structures for the simulation and visualization system.

This header file declares all global macros, extern variables, and data types required across multiple modules for task management, visualization, and perception in a 2D simulation environment.

7.3.2 Tasks Constants

- Defines task periods (in milliseconds):
 - PERCEPTION_PERIOD: Period for the perception task.
 - TRAJECTORY_PERIOD: Period for the trajectory planning task.
 - CONTROL_PERIOD: Period for the control task.
 - DISPLAY_PERIOD: Period for the display update task.
- Defines deadlines (in milliseconds), set equal to the corresponding task periods:
 - PERCEPTION_DEADLINE, TRAJECTORY_DEADLINE, CONTROL_DEADLINE, DISPLAY_DEADLINE.
- Defines task priorities (lower number indicates higher priority):
 - PERCEPTION_PRIORITY, TRAJECTORY_PRIORITY, CONTROL_PRIORITY, DISPLAY_PRIORITY.

7.3.3 Drawing Mutex

- Declares the external pthread_mutex_t variable draw_mutex used to synchronize drawing operations across threads.

7.3.4 Conversion Constants

- px_per_meter: Conversion factor from meters to pixels.
- deg2rad: Constant to convert degrees to radians.

7.3.5 Visualization Constants

- Declares external constants and variables for visualization:
 - title: Pointer to a string containing the window title.
 - X_MAX, Y_MAX: Maximum dimensions (in pixels) of the window or display.
 - cone_radius: Radius used for drawing cones.
- Declares external BITMAP pointers for various graphical assets:
 - control_panel: Bitmap for the control panel.
 - steering_wheel: Bitmap for the steering wheel graphic.
 - throttle_gauge: Bitmap for the throttle gauge.
 - background: Background image bitmap.
 - track: Bitmap representing the track.
 - car: Bitmap representing the car.
 - perception: Bitmap used for displaying perception data.
 - trajectory_bmp: Bitmap for trajectory visualization.
 - display_buffer: Bitmap used as a back buffer for display updates.
- Defines maxThrottleHeight: Maximum height for the throttle gauge.
- Declares external color variables for various color schemes used in the display:
 - grass_green, asphalt_gray, white, pink, yellow, blue.

7.3.6 Car Pose

- Declares global variables representing the car's state:
 - car_x, car_y: The car's x and y positions in meters.
 - car_angle: The angle of the car in degrees.

7.3.7 Perception Constants

- MAX_DETECTED_CONES: Macro defining the maximum number of cones that can be detected simultaneously.
- [pointcloud_t](#): Struct representing a detected cone with the following fields:
 - point_x: x-coordinate of the detected point.
 - point_y: y-coordinate of the detected point.
 - distance: Distance from the sensor to the cone.
 - color: Color identifier for the cone.
- Declares an external array 'measures' of [pointcloud_t](#) to hold the detected cone data.
- Declares a semaphore 'lidar_sem' used to synchronize the passing of LiDAR data between tasks.

7.3.8 Macro Definition Documentation

7.3.8.1 CONTROL_DEADLINE

```
#define CONTROL_DEADLINE CONTROL\_PERIOD
```

7.3.8.2 CONTROL_ID

```
#define CONTROL_ID 3
```

7.3.8.3 CONTROL_PERIOD

```
#define CONTROL_PERIOD 50
```

7.3.8.4 CONTROL_PRIORITY

```
#define CONTROL_PRIORITY 5
```

7.3.8.5 deg2rad

```
#define deg2rad 0.017453292519943295769236907684886f
```

7.3.8.6 DISPLAY_DEADLINE

```
#define DISPLAY_DEADLINE DISPLAY\_PERIOD
```

7.3.8.7 DISPLAY_ID

```
#define DISPLAY_ID 4
```

7.3.8.8 DISPLAY_PERIOD

```
#define DISPLAY_PERIOD 17
```

7.3.8.9 DISPLAY_PRIORITY

```
#define DISPLAY_PRIORITY 30
```

7.3.8.10 MAX_CONES_MAP

```
#define MAX_CONES_MAP 3000
```

7.3.8.11 MAX_DETECTED_CONES

```
#define MAX_DETECTED_CONES 360
```

7.3.8.12 maxThrottleHeight

```
#define maxThrottleHeight 100
```

7.3.8.13 PERCEPTION_DEADLINE

```
#define PERCEPTION_DEADLINE PERCEPTION\_PERIOD
```

7.3.8.14 PERCEPTION_ID

```
#define PERCEPTION_ID 1
```

7.3.8.15 PERCEPTION_PERIOD

```
#define PERCEPTION_PERIOD 50
```

7.3.8.16 PERCEPTION_PRIORITY

```
#define PERCEPTION_PRIORITY 15
```

7.3.8.17 PROFILING

```
#define PROFILING
```

7.3.8.18 px_per_meter

```
#define px_per_meter 100
```

7.3.8.19 SETTINGS_DEADLINE

```
#define SETTINGS_DEADLINE SETTINGS\_PERIOD
```

7.3.8.20 SETTINGS_ID

```
#define SETTINGS_ID 5
```

7.3.8.21 SETTINGS_PERIOD

```
#define SETTINGS_PERIOD 100
```

7.3.8.22 SETTINGS_PRIORITY

```
#define SETTINGS_PRIORITY 35
```

7.3.8.23 TRAJECTORY_DEADLINE

```
#define TRAJECTORY_DEADLINE TRAJECTORY\_PERIOD
```

7.3.8.24 TRAJECTORY_ID

```
#define TRAJECTORY_ID 2
```

7.3.8.25 TRAJECTORY_PERIOD

```
#define TRAJECTORY_PERIOD 100
```

7.3.8.26 TRAJECTORY_PRIORITY

```
#define TRAJECTORY_PRIORITY 20
```

7.3.9 Variable Documentation

7.3.9.1 asphalt_gray

```
int asphalt_gray
```

7.3.9.2 background

```
background [extern]
```

-

7.3.9.3 blue

```
grass_green asphalt_gray white pink yellow blue
```

Integer values representing various colors used for rendering the simulation's elements.

- LiDAR Data:

—

7.3.9.4 car

```
car [extern]
```

-

7.3.9.5 car_angle

```
car_angle [extern]
```

The initial orientation angle of the car (in degrees).

- Graphical Resources:

—

7.3.9.6 car_mutex

```
pthread_mutex_t car_mutex [extern]
```

7.3.9.7 car_x

```
car_x [extern]
```

The initial X-coordinate position of the car within the simulation.

-

7.3.9.8 car_y

```
car_y
```

The initial Y-coordinate position of the car within the simulation.

-

7.3.9.9 cone_radius

```
cone_radius [extern]
```

The radius (in meters) for cone-related calculations, set to 0.05.

- Car Initial State:

—

7.3.9.10 cones

```
cone cones[MAX_CONES_MAP] [extern]
```


7.3.9.11 control_panel

```
control_panel [extern]
```

-

7.3.9.12 display_buffer

```
display_buffer [extern]
```

Bitmap pointers used for rendering various components of the simulation such as the background, track markings, car image, perception overlays, trajectory visualizations, and the overall display buffer.

- Color Definitions:

—

7.3.9.13 dl_miss_control

```
int dl_miss_control [extern]
```

7.3.9.14 dl_miss_display

```
int dl_miss_display [extern]
```

7.3.9.15 dl_miss_perception

```
int dl_miss_perception [extern]
```

7.3.9.16 dl_miss_settings

```
int dl_miss_settings [extern]
```

7.3.9.17 dl_miss_trajectory

```
int dl_miss_trajectory [extern]
```

7.3.9.18 draw_mutex

```
draw_mutex [extern]
```

A POSIX mutex that ensures mutual exclusion during drawing operations.

Note

The multiplier 'px_per_meter' used for computing X_MAX and Y_MAX is defined elsewhere.

7.3.9.19 grass_green

```
int grass_green [extern]
```

7.3.9.20 green

```
int green
```

7.3.9.21 lidar_sem

```
lidar_sem [extern]
```

A POSIX semaphore used to synchronize access to LiDAR data.

•

7.3.9.22 map_mutex

```
pthread_mutex_t map_mutex [extern]
```

7.3.9.23 measures

`measures` [extern]

An array of 360 LiDAR measurements (one per degree) representing the simulated point cloud data.

- Thread Synchronization:

—

7.3.9.24 perception

`perception` [extern]

-

7.3.9.25 pink

`int pink`

7.3.9.26 red

`int red` [extern]

7.3.9.27 starting_cone_positions

`cone` `starting_cone_positions`[`MAX_CONES_MAP`] [extern]

Store the default cone positions before collisions

7.3.9.28 steering_wheel

`steering_wheel` [extern]

-

7.3.9.29 throttle_gauge

`throttle_gauge` [extern]

Bitmap pointers for UI elements including the control panel, steering wheel, and throttle gauge.

-

7.3.9.30 title

`title` [extern]

The simulation title ("2D FSAE sim by rimaturus").

- Simulation Dimensions:

—

7.3.9.31 track

`track` [extern]

-

7.3.9.32 trajectory_bmp

`trajectory_bmp` [extern]

-

7.3.9.33 white

`int white`

7.3.9.34 X_MAX

X_MAX [extern]

The maximum horizontal dimension of the simulation in pixels, computed as $19 * \text{px_per_meter}$.

-

7.3.9.35 Y_MAX

Y_MAX [extern]

The maximum vertical dimension of the simulation in pixels, computed as $12 * \text{px_per_meter}$.

- Physics and Geometry:

-

7.3.9.36 yellow

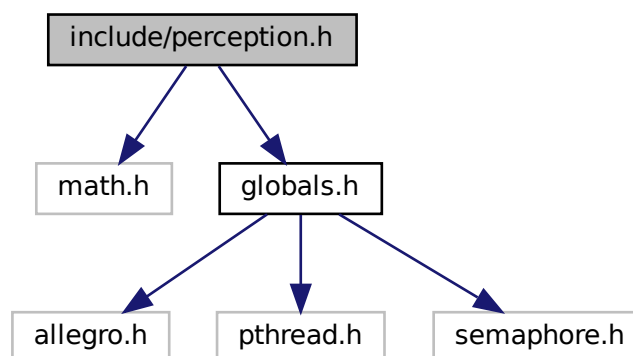
int yellow [extern]

7.4 include/perception.h File Reference

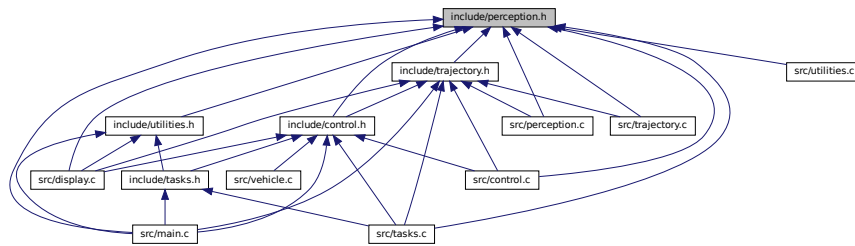
Header file for perception-related types, constants, and function declarations.

```
#include <math.h>
#include "globals.h"
```

Include dependency graph for perception.h:



This graph shows which files directly or indirectly include this file:



Classes

- struct [cone_border](#)
Represents a border of a cone as seen in a LiDAR scan.
- struct [candidate_cone](#)
Represents a candidate cone that might be validated after multiple detections.
- struct [Hough_circle_point_t](#)
Represents a point in the Hough circle transformation space.

Macros

- `#define` [MAX_POINTS_PER_CONE](#) 180
Processes the LiDAR measurements.
- `#define` [maxRange](#) 2.0f
- `#define` [MAX_CANDIDATES](#) 100000
- `#define` [DETECTIONS_THRESHOLD](#) 10

Functions

- void [lidar](#) ([pointcloud_t](#) *measures)
- void [mapping](#) ([cone](#) *detected_cones)
- void [check_nearest_point](#) (int angle, float new_point_x, float new_point_y, int color, [cone_border](#) *cone_borders)
- void [update_map](#) ([cone](#) *detected_cones)

Variables

- const float [ignore_distance](#)
- const int [sliding_window](#)
- const int [angle_step](#)
- int [start_angle](#)
- [cone](#) detected_cones [[MAX_DETECTED_CONES](#)]
- [cone](#) track_map [[MAX_CONES_MAP](#)]
- int [track_map_idx](#)

7.4.1 Detailed Description

Header file for perception-related types, constants, and function declarations.

This file contains the datatypes and function prototypes used for perception in the 2D simulation. It includes structures to represent cones, cone borders, candidate cones, and mapping functionalities such as LiDAR scanning, real-time mapping, and track map updating.

Global constants and configurable parameters:

- `MAX_POINTS_PER_CONE`: Maximum number of points (angles) that can define the border of a cone.
- `MAX_CONES_MAP`: Maximum number of cones that can be stored in the track map.
- `maxRange`: Maximum detection range (in meters) for the LiDAR.
- `ignore_distance`: Distance threshold below which detected points may be ignored.
- `sliding_window`, `angle_step`, `start_angle`: Parameters that control the LiDAR scanning and cone detection algorithm.

Global arrays:

- `detected_cones`: Array containing the cones detected by perception.
- `track_map`: Array representing the global track map of cones.
- `track_map_idx`: Current index in the `track_map` array.

Other constants:

- `MAX_CANDIDATES`: Maximum number of candidate cones tracked.
- `DETECTIONS_THRESHOLD`: Minimum number of detections required to consider a candidate cone valid.

7.4.2 Macro Definition Documentation

7.4.2.1 DETECTIONS_THRESHOLD

```
#define DETECTIONS_THRESHOLD 10
```

7.4.2.2 MAX_CANDIDATES

```
#define MAX_CANDIDATES 100000
```

7.4.2.3 MAX_POINTS_PER_CONE

```
#define MAX_POINTS_PER_CONE 180
```

Processes the LiDAR measurements.

This function simulates or processes a LiDAR scan, updating the point cloud information based on the car's current position.

Parameters

<i>car_x</i>	The X coordinate of the car's position.
<i>car_y</i>	The Y coordinate of the car's position.
<i>measures</i>	Pointer to the point cloud structure to be updated with the LiDAR measurements.

Performs real-time mapping using detected cones.

This function updates the array of detected cones by correlating the car's current position and orientation with cone positions.

Parameters

<i>car_x</i>	The X coordinate of the car's position.
<i>car_y</i>	The Y coordinate of the car's position.
<i>car_angle</i>	The orientation angle of the car (in degrees or radians as defined elsewhere).
<i>detected_cones</i>	Pointer to the array where detected cones are stored.

Checks and updates the nearest point for a given LiDAR scan angle.

This function compares a new detected point against existing data to determine if it belongs to a known cone border, updating the cone border data accordingly.

Parameters

<i>angle</i>	The scan angle index from the LiDAR measurement.
<i>new_point_x</i>	The X coordinate of the new detected point.
<i>new_point_y</i>	The Y coordinate of the new detected point.
<i>color</i>	The color associated with the new detected point.
<i>cone_borders</i>	Pointer to the cone_border structure to be updated.

Updates the global track map with the latest detected cones.

This function takes the newly detected cones and integrates them into the overall track map, which represents the accumulated perception of the environment.

Parameters

<i>detected_cones</i>	Pointer to the array of cones that have been newly detected.
-----------------------	--

7.4.2.4 maxRange

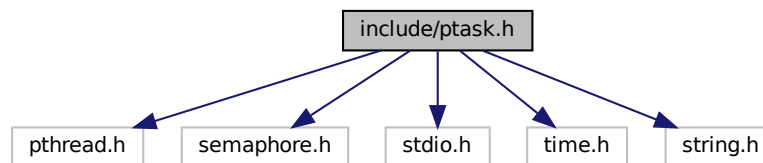
```
#define maxRange 2.0f
```


7.5 include/ptask.h File Reference

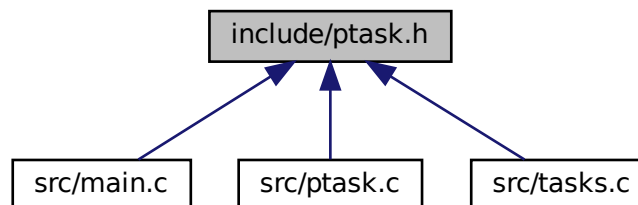
Interface for managing periodic tasks and time utilities.

```
#include <pthread.h>
#include <semaphore.h>
#include <stdio.h>
#include <time.h>
#include <string.h>
```

Include dependency graph for ptask.h:



This graph shows which files directly or indirectly include this file:



Classes

- struct [timespec_custom](#)
- struct [task_par](#)

Macros

- #define [_POSIX_C_SOURCE](#) 200809L
- #define [MAX_TASKS](#) 32
- #define [MICRO](#) 1
- #define [NANO](#) 2
- #define [ACT](#) 1
- #define [DEACT](#) 0

Typedefs

- typedef struct [timespec_custom](#) [timespec_custom](#)
- typedef struct [task_par](#) [task_par](#)

Functions

- void [time_copy](#) ([timespec_custom](#) *td, [timespec_custom](#) ts)
- void [time_add_ms](#) ([timespec_custom](#) *t, int ms)
- int [time_cmp](#) ([timespec_custom](#) t1, [timespec_custom](#) t2)
- void [ptask_init](#) (int policy)
- long [get_systime](#) (int unit)
- int [task_create](#) (int i, void *(*task)(void *), int period, int drel, int prio, int aflag)
- int [get_task_index](#) (void *arg)
- void [wait_for_activation](#) (int i)
- void [task_activate](#) (int i)
- int [deadline_miss](#) (int i)
- void [wait_for_period](#) (int i)
- void [task_set_period](#) (int i, int per)
- void [task_set_deadline](#) (int i, int drel)
- int [task_period](#) (int i)
- int [task_deadline](#) (int i)
- int [task_dmiss](#) (int i)
- void [task_atime](#) (int i, [timespec_custom](#) *at)
- void [task_adline](#) (int i, [timespec_custom](#) *dl)
- void [wait_for_task_end](#) (int i)

Variables

- [task_par](#) tp [MAX_TASKS]
- [timespec_custom](#) ptask_t0
- int [ptask_policy](#)

7.5.1 Detailed Description

Interface for managing periodic tasks and time utilities.

This header file defines data types, macros, global variables, and function prototypes that collectively support a periodic real-time task management system.

The API supports:

- Configuring a maximum number of tasks via [MAX_TASKS](#).
- Defining time units using MICRO (for microseconds) and NANO (for nanoseconds).
- Controlling task activation state with the ACT and DEACT flags.
- Using a custom time structure ([timespec_custom](#)) with the same layout as struct timespec, enabling compatibility with standard POSIX time functions (e.g., clock_gettime, clock_nanosleep).

Two primary data structures are provided:

- [timespec_custom](#): A custom time structure for representing seconds and nanoseconds.
- [task_par](#): A structure representing a periodic task that contains the task identifier, period (in milliseconds), relative deadline (in milliseconds), scheduling priority, a counter for deadline misses, activation and deadline timestamps, a thread identifier, and a semaphore for activation control.

Global Variables:

- [tp\[MAX_TASKS\]](#): Array of [task_par](#) structures holding task parameters.
- [ptask_t0](#): Reference starting time for the system.
- [ptask_policy](#): Global scheduling policy to be applied to all tasks.

Provided Functions:

- [time_copy\(\)](#): Copies a [timespec_custom](#) value.
- [time_add_ms\(\)](#): Adds a given number of milliseconds to a [timespec_custom](#) value.
- [time_cmp\(\)](#): Compares two [timespec_custom](#) values, returning 1 if the first is greater, -1 if it is smaller, and 0 if they are equal.
- [ptask_init\(\)](#): Initializes the periodic task system by setting the scheduling policy, capturing the reference system time, and initializing activation semaphores.
- [get_systime\(\)](#): Returns the elapsed system time since initialization in either microseconds or nanoseconds.
- [task_create\(\)](#): Creates a new periodic task with specified period, relative deadline, priority, and activation flag. If activated immediately, the task's semaphore is posted.
- [get_task_index\(\)](#): Retrieves the task index from the task parameter pointer.
- [wait_for_activation\(\)](#): Blocks the task until its activation semaphore is posted. It then updates the task's activation and deadline times based on its period and deadline.
- [task_activate\(\)](#): Posts the semaphore to activate the task.
- [deadline_miss\(\)](#): Checks if the current time has exceeded the task's deadline; if so, it increments the deadline miss counter and returns 1.
- [wait_for_period\(\)](#): Uses absolute time waiting ([clock_nanosleep](#)) to synchronize the task with its defined period, avoiding cumulative time drift.
- Task Parameter Setters and Getters:
 - [task_set_period\(\)](#) and [task_set_deadline\(\)](#): Modify a task's period and relative deadline.
 - [task_period\(\)](#), [task_deadline\(\)](#), and [task_dmiss\(\)](#): Retrieve a task's period, relative deadline, and the count of deadline misses.
 - [task_atime\(\)](#) and [task_adline\(\)](#): Get the next activation time and current deadline time for a task.
- [wait_for_task_end\(\)](#): Waits (joins) for a task thread to complete execution.

Note

The function implementations are only compiled when `PTASK_IMPLEMENTATION` is defined.

Author

Your Name or Organization

Date

YYYY-MM-DD

7.5.2 Macro Definition Documentation

7.5.2.1 `_POSIX_C_SOURCE`

```
#define _POSIX_C_SOURCE 200809L
```

7.5.2.2 `ACT`

```
#define ACT 1
```

7.5.2.3 `DEACT`

```
#define DEACT 0
```

7.5.2.4 `MAX_TASKS`

```
#define MAX_TASKS 32
```

7.5.2.5 `MICRO`

```
#define MICRO 1
```

7.5.2.6 `NANO`

```
#define NANO 2
```

7.5.3 Typedef Documentation

7.5.3.1 task_par

```
typedef struct task_par task_par
```

7.5.3.2 timespec_custom

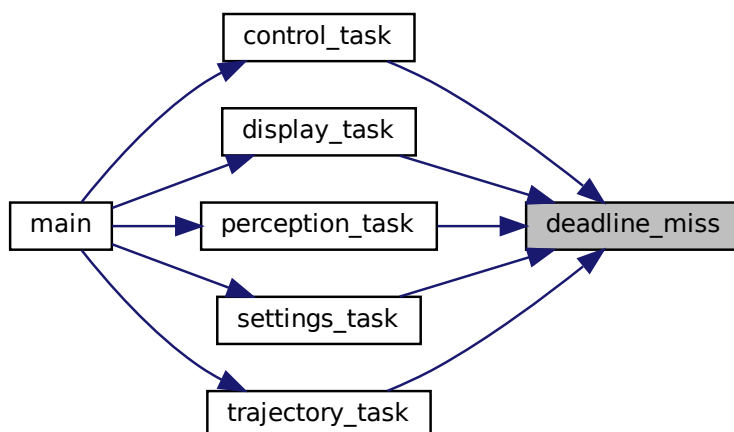
```
typedef struct timespec_custom timespec_custom
```

7.5.4 Function Documentation

7.5.4.1 deadline_miss()

```
int deadline_miss (  
    int i )
```

Here is the caller graph for this function:



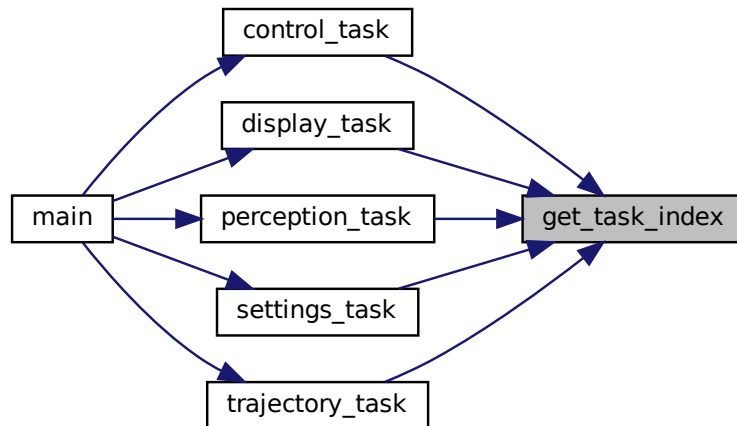
7.5.4.2 get_systime()

```
long get_systime (  
    int unit )
```

7.5.4.3 get_task_index()

```
int get_task_index (
    void * arg )
```

Here is the caller graph for this function:



7.5.4.4 ptask_init()

```
void ptask_init (
    int policy )
```

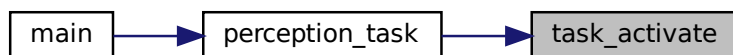
Here is the caller graph for this function:



7.5.4.5 task_activate()

```
void task_activate (
    int i )
```

Here is the caller graph for this function:



7.5.4.6 task_adline()

```
void task_adline (
    int i,
    timespec_custom * dl )
```

7.5.4.7 task_atime()

```
void task_atime (
    int i,
    timespec_custom * at )
```

7.5.4.8 task_create()

```
int task_create (
    int i,
    void (*)(void *) task,
    int period,
    int drel,
    int prio,
    int aflag )
```

Here is the caller graph for this function:



7.5.4.9 task_deadline()

```
int task_deadline (  
    int i )
```

7.5.4.10 task_dmiss()

```
int task_dmiss (  
    int i )
```

7.5.4.11 task_period()

```
int task_period (  
    int i )
```

7.5.4.12 task_set_deadline()

```
void task_set_deadline (  
    int i,  
    int drel )
```

7.5.4.13 task_set_period()

```
void task_set_period (  
    int i,  
    int per )
```

7.5.4.14 time_add_ms()

```
void time_add_ms (  
    timespec_custom * t,  
    int ms )
```


7.5.4.15 time_cmp()

```
int time_cmp (
    timespec_custom t1,
    timespec_custom t2 )
```

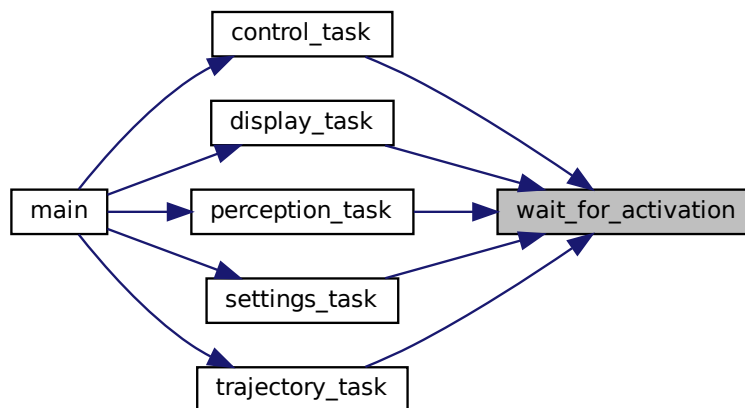
7.5.4.16 time_copy()

```
void time_copy (
    timespec_custom * td,
    timespec_custom ts )
```

7.5.4.17 wait_for_activation()

```
void wait_for_activation (
    int i )
```

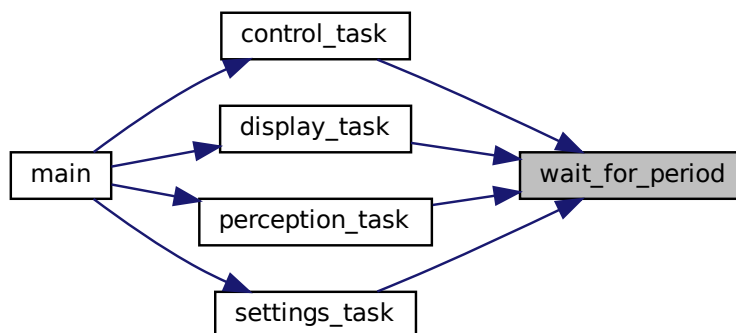
Here is the caller graph for this function:



7.5.4.18 wait_for_period()

```
void wait_for_period (
    int i )
```

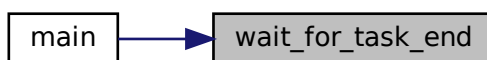
Here is the caller graph for this function:



7.5.4.19 wait_for_task_end()

```
void wait_for_task_end (
    int i )
```

Here is the caller graph for this function:



7.5.5 Variable Documentation

7.5.5.1 ptask_policy

```
int ptask_policy [extern]
```

7.5.5.2 ptask_t0

```
timespec_custom ptask_t0 [extern]
```

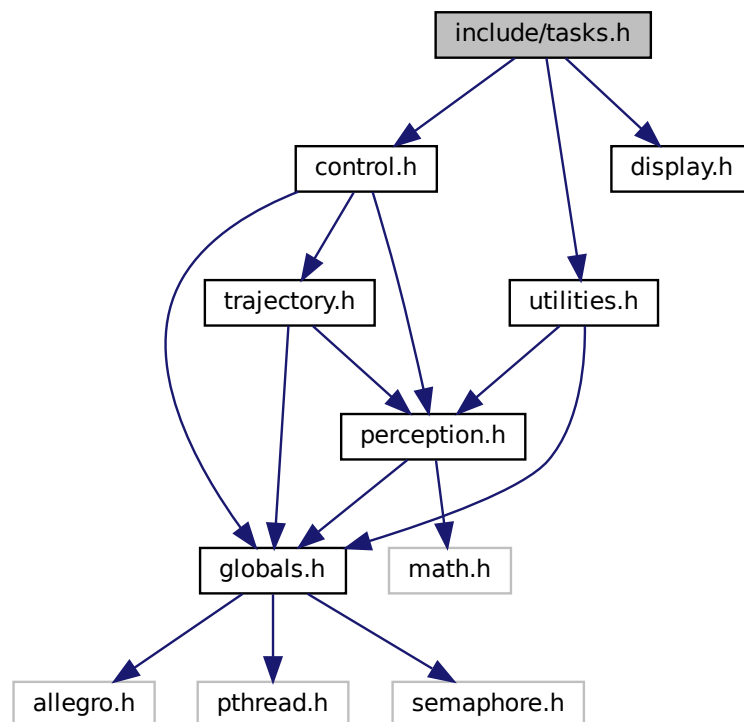
7.5.5.3 tp

```
task_par tp[MAX_TASKS] [extern]
```

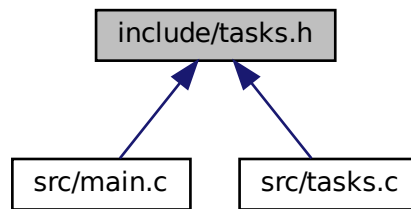
7.6 include/tasks.h File Reference

Declaration of task thread functions for the simulation.

```
#include "utilities.h"  
#include "control.h"  
#include "display.h"  
Include dependency graph for tasks.h:
```



This graph shows which files directly or indirectly include this file:



Functions

- void * [perception_task](#) (void *arg)
Perception task.
- void * [trajectory_task](#) (void *arg)
- void * [control_task](#) (void *arg)
- void * [display_task](#) (void *arg)
- void * [settings_task](#) (void *arg)

7.6.1 Detailed Description

Declaration of task thread functions for the simulation.

This header file declares thread functions for various tasks in the simulation:

- Perception: Processes sensor or simulation data to interpret the environment.
- Trajectory: Computes and updates planned paths or trajectories.
- Control: Implements control logic that may handle both manual and autonomous inputs.
- Display: Manages rendering of the simulation's visual output.

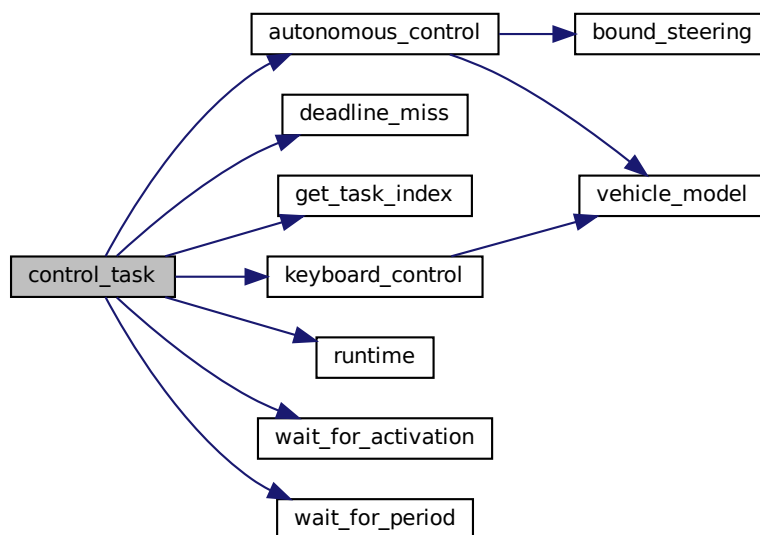
Each task is designed to be executed in its own thread, and the functions follow the prototype required by POSIX thread routines.

7.6.2 Function Documentation

7.6.2.1 control_task()

```
void* control_task (  
    void * arg )
```

Here is the call graph for this function:



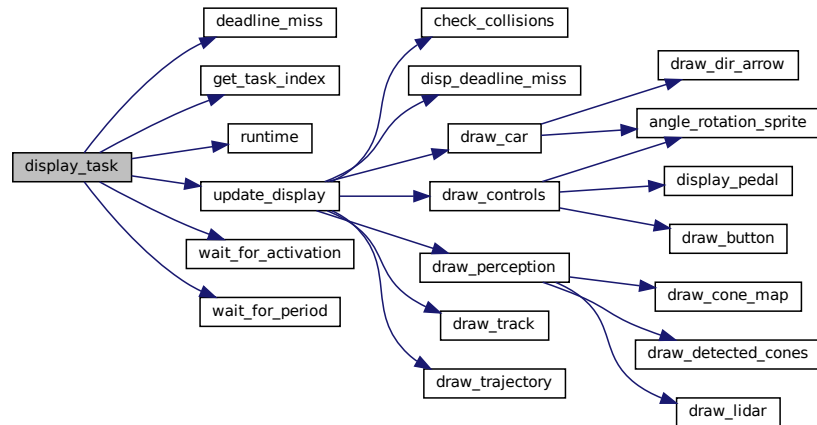
Here is the caller graph for this function:



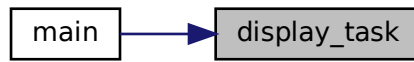
7.6.2.2 display_task()

```
void* display_task (  
    void * arg )
```

Here is the call graph for this function:



Here is the caller graph for this function:



7.6.2.3 perception_task()

```
void* perception_task (
    void * arg )
```

Perception task.

This function executes perception related operations, such as processing sensor data or simulation input to understand the surrounding environment.

Parameters

<i>arg</i>	Pointer to the arguments required by the task. The actual type and usage of this parameter are defined by the implementation context.
------------	---

Returns

Pointer representing the function's outcome, typically not used.

Trajectory planning task.

This function is responsible for calculating and updating the movement trajectory for the entity in simulation. It handles the logic for path planning based on environmental data and simulation dynamics.

Parameters

<i>arg</i>	Pointer to the arguments specific to the trajectory task.
------------	---

Returns

Pointer representing the result of the function, usually unused.

Control task.

This function manages the control logic, determining inputs for either keyboard control or autonomous behavior. It integrates various control signals and decisions, ensuring the entity reacts appropriately within the simulation.

Parameters

<i>arg</i>	Pointer to a structure or value containing control parameters.
------------	--

Returns

Pointer to the result of the control processing, typically not utilized.

Display update task.

This function handles the rendering process for the simulation's trajectory display. It updates the visual output based on the current state of the simulation data.

Parameters

<i>arg</i>	Pointer to display specific parameters or context information.
------------	--

Returns

Pointer indicating the outcome of the display operation, generally unused.

Task managing simulation settings and configuration

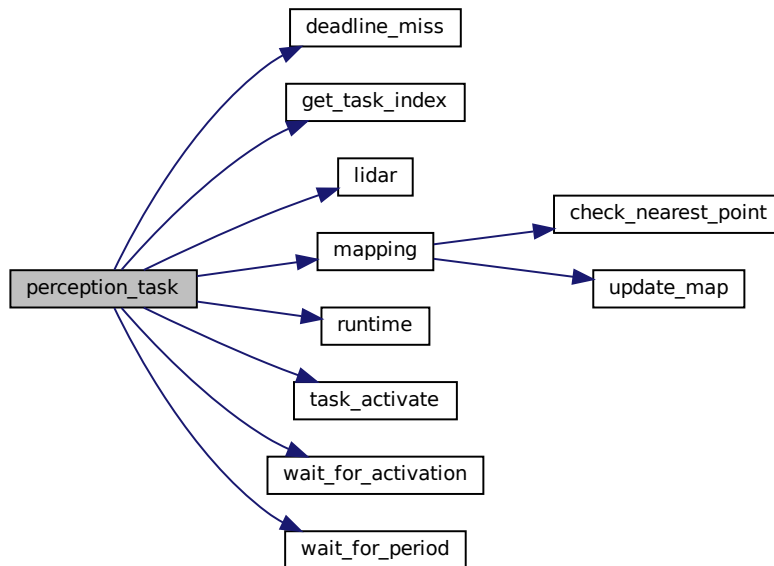
Parameters

<i>arg</i>	Pointer to task arguments (void pointer for thread compatibility)
------------	---

Returns

void* Return value of the thread (unused)

Thread task that handles simulation settings and configuration updates. Should be used with `pthread_create()`. Here is the call graph for this function:



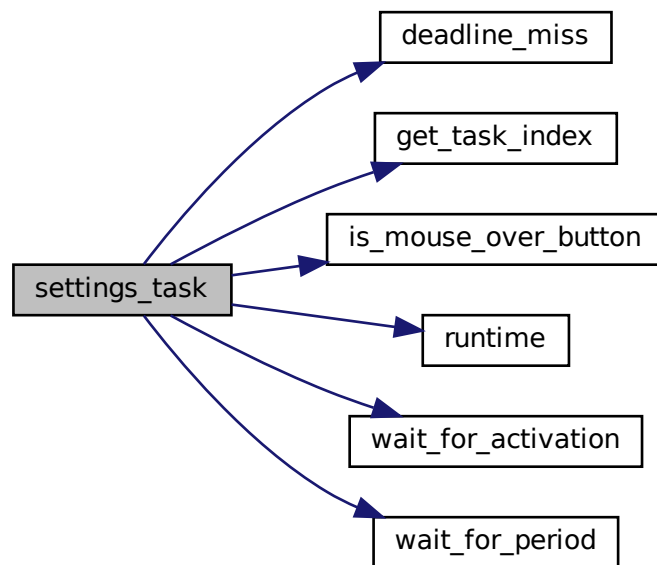
Here is the caller graph for this function:

**7.6.2.4 settings_task()**

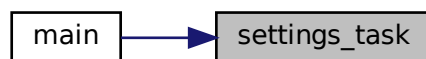
```

void* settings_task (
    void * arg )
  
```


Here is the call graph for this function:



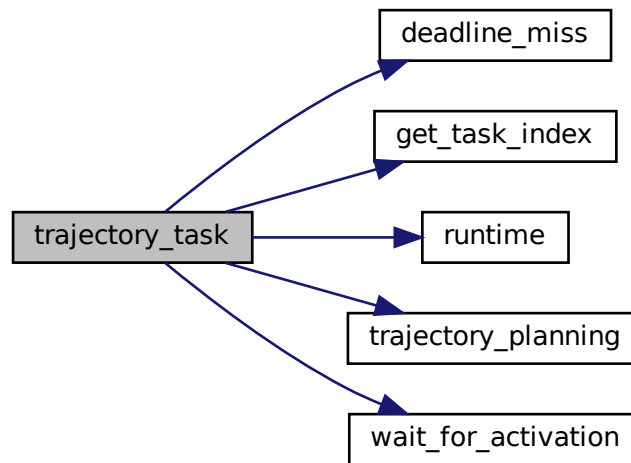
Here is the caller graph for this function:



7.6.2.5 trajectory_task()

```
void* trajectory_task (  
    void * arg )
```

Here is the call graph for this function:



Here is the caller graph for this function:

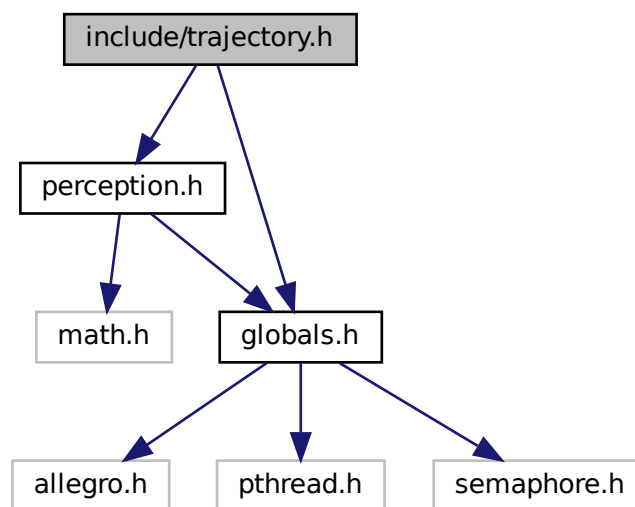


7.7 include/trajectory.h File Reference

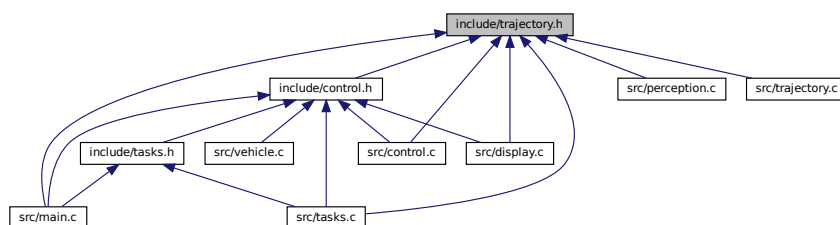
Provides declarations for trajectory planning based on cone detection.

```
#include "perception.h"  
#include "globals.h"
```

Include dependency graph for trajectory.h:



This graph shows which files directly or indirectly include this file:



Classes

- struct [waypoint](#)

Plans the trajectory for the car based on its current state and detected obstacles.

Macros

- `#define` [MAX_WAYPOINTS](#) (40*[MAX_DETECTED_CONES](#))

Functions

- void [trajectory_planning](#) ([waypoint](#) *trajectory)

Variables

- `waypoint trajectory` [MAX_WAYPOINTS]
Global array of waypoints forming the planned trajectory.
- `int trajectory_idx`
Global index used to track the number of waypoints currently stored in the trajectory.

7.7.1 Detailed Description

Provides declarations for trajectory planning based on cone detection.

This header defines the data structures and function prototypes necessary for generating trajectories (waypoints) that guide the car through detected cones.

7.7.2 Macro Definition Documentation

7.7.2.1 MAX_WAYPOINTS

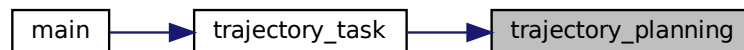
```
#define MAX_WAYPOINTS (40*MAX_DETECTED_CONES)
```

7.7.3 Function Documentation

7.7.3.1 trajectory_planning()

```
void trajectory_planning (  
    waypoint * trajectory )
```

Here is the caller graph for this function:



7.7.4 Variable Documentation

7.7.4.1 trajectory

```
trajectory [extern]
```

Global array of waypoints forming the planned trajectory.

This array stores the computed trajectory based on the detected cones. It is sized to accommodate up to twice the number of maximum detectable cones ($2 * \text{MAX_DETECTED_CONES}$).

7.7.4.2 trajectory_idx

```
trajectory_idx [extern]
```

Global index used to track the number of waypoints currently stored in the trajectory.

This variable indicates the current position in the trajectory array, facilitating the addition of new waypoints.

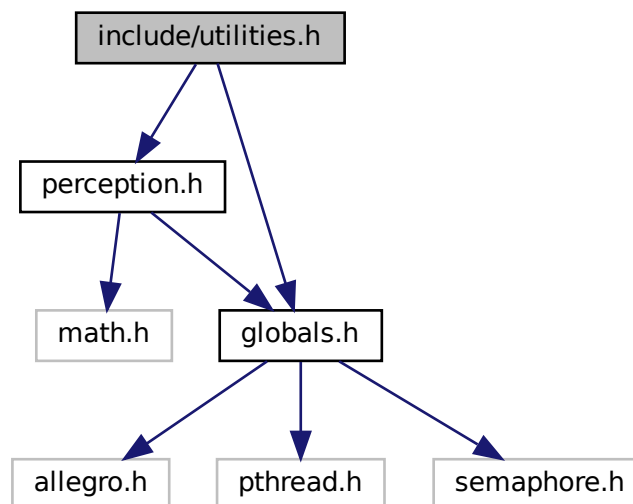
7.8 include/utilities.h File Reference

Utility function prototypes for cone management and runtime tasks.

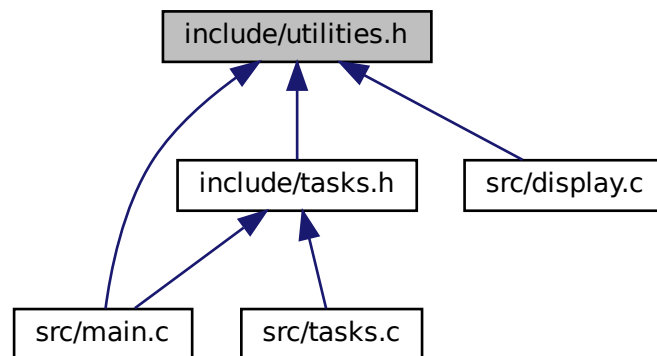
```
#include <perception.h>
```

```
#include <globals.h>
```

Include dependency graph for utilities.h:



This graph shows which files directly or indirectly include this file:



Functions

- void `init_cones` (`cone *cones`)
Initializes an array of cone structures.
- void `load_cones_positions` (`const char *filename`, `cone *cones`, `int max_cones`)
- float `angle_rotation_sprite` (`float angle`)
- void `runtime` (`int stop_signal`, `char *task_name`)

7.8.1 Detailed Description

Utility function prototypes for cone management and runtime tasks.

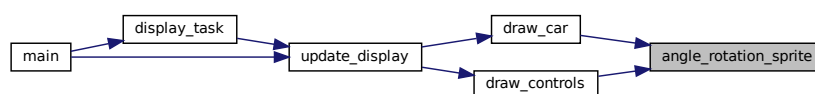
This header declares functions for initializing cone structures, loading cone positions from a file, calculating sprite rotations, and handling runtime processes.

7.8.2 Function Documentation

7.8.2.1 `angle_rotation_sprite()`

```
float angle_rotation_sprite (
    float angle )
```

Here is the caller graph for this function:



7.8.2.2 init_cones()

```
void init_cones (
    cone * cones )
```

Initializes an array of cone structures.

This function sets up the specified array of cones so that each cone is properly initialized.

Parameters

<i>cones</i>	Pointer to the first element of an array of cone structures.
--------------	--

Loads cone positions from a file.

Reads cone position data from the provided file and populates the cone array accordingly. Only up to `max_cones` entries are loaded, ensuring the array bounds are respected.

Parameters

<i>filename</i>	Path to the file containing cone position data.
<i>cones</i>	Pointer to an array of cone structures where the positions will be stored.
<i>max_cones</i>	The maximum number of cones to load from the file.

Computes the sprite rotation angle.

Calculates and returns a new rotation angle for a sprite based on the input `angle`. The calculation may involve normalization or other transformations suitable for sprite handling.

Parameters

<i>angle</i>	The initial angle value (in degrees or radians, as defined by the application's convention).
--------------	--

Returns

The adjusted angle after applying the rotation transformation.

Handles runtime processes based on a stop signal.

Executes runtime tasks, potentially including logging or task management, until a stop condition defined by `stop_signal` is met. The associated task description is provided via `task_name`.

Parameters

<i>stop_signal</i>	Signal value indicating when to terminate the runtime process.
<i>task_name</i>	A string identifier for the task being executed.

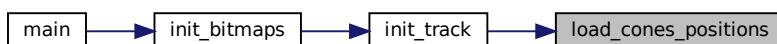
Here is the caller graph for this function:



7.8.2.3 load_cones_positions()

```
void load_cones_positions (
    const char * filename,
    cone * cones,
    int max_cones )
```

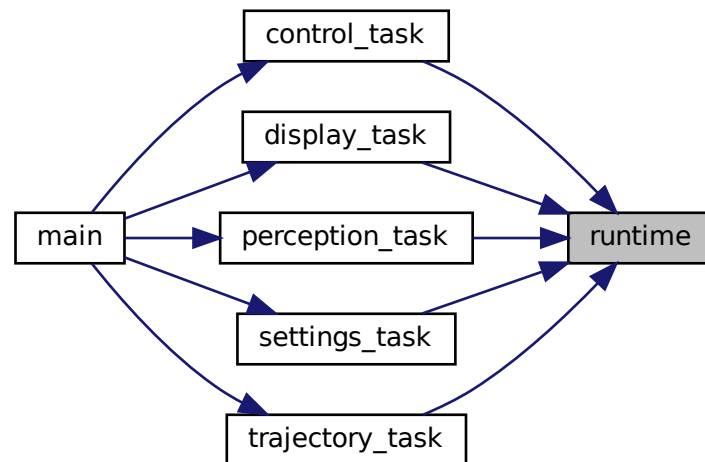
Here is the caller graph for this function:



7.8.2.4 runtime()

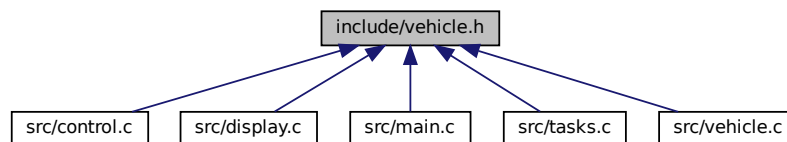
```
void runtime (
    int stop_signal,
    char * task_name )
```


Here is the caller graph for this function:



7.9 include/vehicle.h File Reference

This graph shows which files directly or indirectly include this file:



Functions

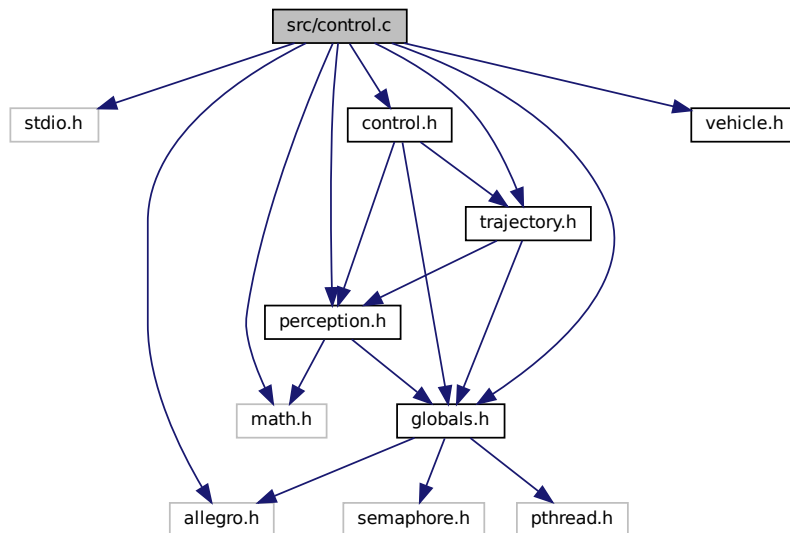
- void [vehicle_model](#) (float [pedal](#), float [steering](#))
Simulates the vehicle's movement by updating its position and orientation.
- void [check_collisions](#) ()

7.10 readme.md File Reference

7.11 src/control.c File Reference

Implements vehicle control functions including both manual (keyboard) and autonomous (centerline based) control.

```
#include <stdio.h>
#include <math.h>
#include <allegro.h>
#include "trajectory.h"
#include "perception.h"
#include "globals.h"
#include "control.h"
#include "vehicle.h"
Include dependency graph for control.c:
```



Functions

- void [keyboard_control](#) ()
Adjusts vehicle controls based on keyboard input.
- void [autonomous_control](#) ([waypoint](#) *center_waypoints)
Autonomous control routine using centerline waypoints.
- float [bound_steering](#) (float steering_rads)

Variables

- [waypoint reordered_ahead](#) [MAX_WAYPOINTS]
- int [wp_ahead_idx](#) = 0
- float [pedal](#) = 0.0f
- float [steering](#) = 0.0f
Controls the vehicle using keyboard input.

7.11.1 Detailed Description

Implements vehicle control functions including both manual (keyboard) and autonomous (centerline based) control.

This file contains methods to update the vehicle's motion state. The manual control routine ([keyboard_control](#)) processes user keystrokes to adjust speed ([pedal](#)) and steering angle, while the autonomous control routine uses a centerline of waypoints to compute a reference trajectory and derive the appropriate steering correction.

7.11.2 Function Documentation

7.11.2.1 autonomous_control()

```
void autonomous_control (
    waypoint * center_waypoints )
```

Autonomous control routine using centerline waypoints.

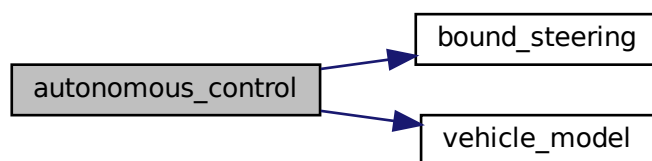
Implements an autonomous control strategy based on a provided centerline represented by an array of waypoints. The function follows these steps:

- Counts the number of valid centerline waypoints (terminated when a waypoint with $x < 0.0f$ is found).
- Filters the centerline to extract waypoints that are located ahead of the vehicle using the `is_in_front` helper.
- Identifies the closest valid waypoint ahead, and depending on the availability of neighboring points, computes a reference trajectory vector. This is done in one of three ways:
 - If at least three valid ahead waypoints exist, the reference vector is computed from the previous to the next waypoint surrounding the closest ahead waypoint.
 - If there are only one or two ahead points (but at least two total waypoints), the reference vector is derived by combining the vector from the vehicle to the last waypoint and the segment between the last two waypoints.
 - If only one waypoint is available, the vector from the vehicle to that waypoint is used as the reference.
- The computed reference trajectory is normalized. If normalization fails, a default forward direction is used.
- The vehicle's current heading is computed as a unit vector based on `local_car_angle`.
- The required steering correction (delta) is obtained by computing the sine of the angle difference through the 2D cross product between the normalized reference vector and the heading vector.
- A constant pedal value is applied.
- Finally, the updated control signals (pedal and delta for steering) are applied to the vehicle by calling `vehicle_model`.

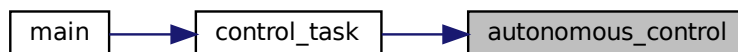
Parameters

in	<i>center_waypoints</i>	Pointer to an array of waypoints representing the desired centerline trajectory. The array should be terminated by a waypoint with $x < 0.0f$.
----	-------------------------	---

Here is the call graph for this function:



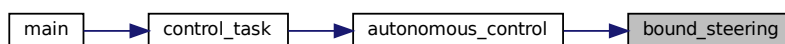
Here is the caller graph for this function:



7.11.2.2 bound_steering()

```
float bound_steering (  
    float steering_rads )
```

Here is the caller graph for this function:



7.11.2.3 keyboard_control()

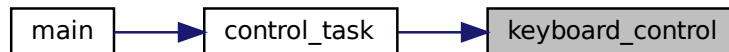
```
void keyboard_control ( )
```

Adjusts vehicle controls based on keyboard input.

This function processes the state of keyboard keys to adjust the vehicle's pedal (speed) and steering angle. It increments or decrements the pedal value and steering angle within pre-defined limits depending on the keys pressed (e.g., KEY_UP, KEY_DOWN, KEY_LEFT, KEY_RIGHT). After updating these control signals, the vehicle's state is updated by calling the `vehicle_model` function. Here is the call graph for this function:



Here is the caller graph for this function:



7.11.3 Variable Documentation

7.11.3.1 pedal

```
float pedal = 0.0f
```

7.11.3.2 reordered_ahead

```
waypoint reordered_ahead[MAX_WAYPOINTS]
```

7.11.3.3 steering

```
float steering = 0.0f
```

Controls the vehicle using keyboard input.

Processes user keyboard inputs to modify the vehicle's position and orientation. The function updates the car's x and y coordinates, as well as its angular orientation (in degrees).

Parameters

<i>car_x</i>	Pointer to the car's x-coordinate.
<i>car_y</i>	Pointer to the car's y-coordinate.
<i>car_angle</i>	Pointer to the car's current angle (in degrees).

Controls the vehicle autonomously along a given trajectory.

Uses the provided trajectory (a sequence of waypoints) to automatically adjust the vehicle's steering and pedal inputs. This function updates the car's position and orientation by computing the necessary control actions based on the navigation path.

Parameters

<i>car_x</i>	Pointer to the car's x-coordinate.
<i>car_y</i>	Pointer to the car's y-coordinate.
<i>car_angle</i>	Pointer to the car's current orientation angle (in degrees).
<i>trajectory</i>	Pointer to the waypoint structure that defines the trajectory.

7.11.3.4 wp_ahead_idx

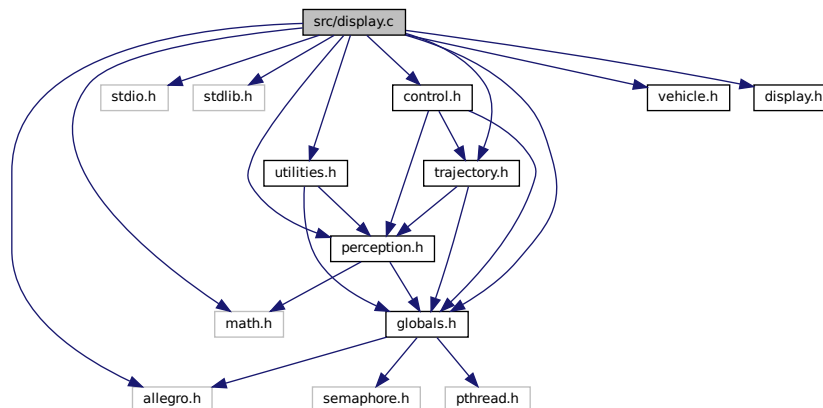
```
int wp_ahead_idx = 0
```

7.12 src/display.c File Reference

Rendering and display update functions for the 2D simulation using Allegro.

```
#include <allegro.h>
#include <math.h>
#include <stdio.h>
#include <stdlib.h>
#include "globals.h"
#include "perception.h"
#include "trajectory.h"
#include "utilities.h"
#include "control.h"
#include "vehicle.h"
#include "display.h"
```

Include dependency graph for display.c:



Functions

- void [draw_dir_arrow](#) ()
Draws a directional arrow representing the car's orientation.
- void [draw_car](#) (float [car_x](#), float [car_y](#), int [car_angle](#))
Renders the car sprite rotated to its current angle.
- void [draw_track](#) ()
Draws the track.
- void [draw_lidar](#) ([pointcloud_t](#) *[measures](#))
Draws the lidar/perception view.
- void [draw_detected_cones](#) ([cone](#) *[detected_cones](#))
Renders the detected cones on the perception bitmap.
- void [draw_cone_map](#) ([cone](#) *[track_map](#), int [track_map_idx](#))
Draws the cone map overlay.
- void [draw_perception](#) ()
Renders the complete perception layer.
- void [draw_trajectory](#) ([waypoint](#) *[trajectory](#))
Draws the trajectory of the car.
- void [draw_controls](#) ()
Draws user control indicators.
- void [update_display](#) ()
Updates the entire display.
- void [display_pedal](#) ([BITMAP](#) *[bitmap](#))
- void [draw_button](#) ([Button](#) *[btn](#), [BITMAP](#) *[buffer](#))
- void [disp_deadline_miss](#) ()

Variables

- int [btn_state_cones](#) = 0
- int [btn_state_perception](#) = 1
- int [btn_state_map](#) = 1
- int [btn_state_traj](#) = 1
- int [btn_state_autonomous](#) = 0
- [Button](#) [buttons](#) [[num_buttons](#)]
Periodic perception task.

7.12.1 Detailed Description

Rendering and display update functions for the 2D simulation using Allegro.

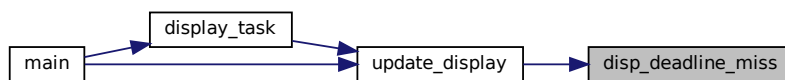
This file contains functions responsible for drawing various elements of the simulation, including the car sprite, direction indicator, track, lidar/perception display, trajectory, and user controls (e.g., steering wheel and pedal gauge). It also handles the final composition of the display buffer and its transfer to the screen.

7.12.2 Function Documentation

7.12.2.1 `disp_deadline_miss()`

```
void disp_deadline_miss ( )
```

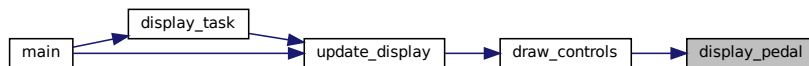
Here is the caller graph for this function:



7.12.2.2 `display_pedal()`

```
void display_pedal (
    BITMAP * bitmap )
```

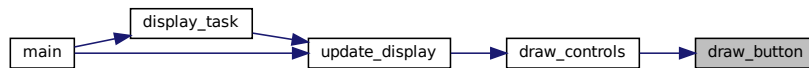
Here is the caller graph for this function:



7.12.2.3 draw_button()

```
void draw_button (
    Button * btn,
    BITMAP * buffer )
```

Here is the caller graph for this function:



7.12.2.4 draw_car()

```
void draw_car (
    float car_x,
    float car_y,
    int car_angle )
```

Renders the car sprite rotated to its current angle.

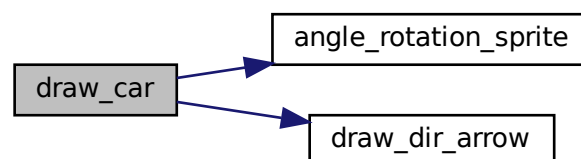
Draws a car shape on the display.

Calculates the proper position for the car sprite based on its coordinates in the world space (scaled by pixels per meter) and draws it using Allegro's `rotate_scaled_sprite` function. In DEBUG mode, the function also draws view angle lines to indicate the car's field of view.

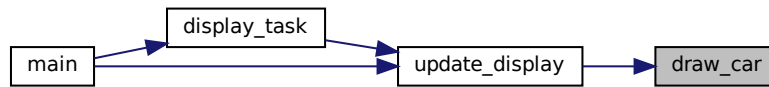
Parameters

<i>car_x</i>	The car's x-coordinate in world units.
<i>car_y</i>	The car's y-coordinate in world units.
<i>car_angle</i>	The car's angle in degrees.

Here is the call graph for this function:



Here is the caller graph for this function:



7.12.2.5 draw_cone_map()

```
void draw_cone_map (
    cone * track_map,
    int track_map_idx )
```

Draws the cone map overlay.

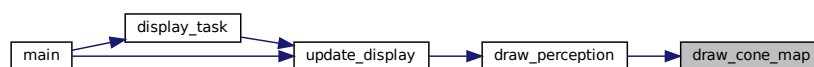
Draws the cone map (track map).

Iterates through the track map data and draws a white filled circle at each cone's position, properly mapping simulation world coordinates to the perception window.

Parameters

<i>track_map</i>	Pointer to an array of cones representing the track map.
<i>track_map_idx</i>	The number of valid entries in the track map array.

Here is the caller graph for this function:

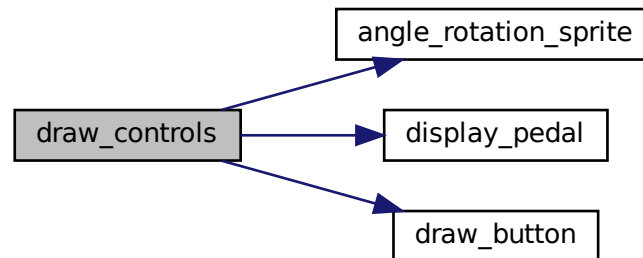


7.12.2.6 draw_controls()

```
void draw_controls ( )
```

Draws user control indicators.

Renders the steering wheel sprite with the correct rotation and draws a gauge for pedal control. The gauge's fill color indicates the pedal level (green for positive, red for non-positive). Here is the call graph for this function:



Here is the caller graph for this function:



7.12.2.7 draw_detected_cones()

```
void draw_detected_cones (
    cone * detected_cones )
```

Renders the detected cones on the perception bitmap.

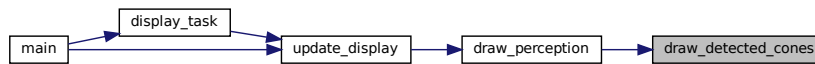
Draws detected cones.

Iterates through the array of detected cones and draws a filled circle at each cone's location, after mapping the coordinates from the simulation world to the perception window.

Parameters

<code>detected_cones</code>	Pointer to an array of detected cone structures.
-----------------------------	--

Here is the caller graph for this function:



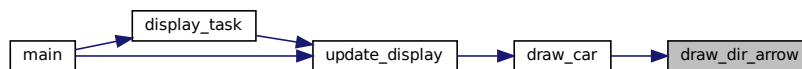
7.12.2.8 draw_dir_arrow()

```
void draw_dir_arrow ( )
```

Draws a directional arrow representing the car's orientation.

Draws a directional arrow.

Computes the arrow's start (at the car's position) and end points based on the car angle, using cosine and sine functions. The arrow shaft and head are drawn as thick green lines, with the head drawn at two angles to form a head shape. Here is the caller graph for this function:



7.12.2.9 draw_lidar()

```
void draw_lidar (
    pointcloud_t * measures )
```

Draws the lidar/perception view.

Draws lidar measurements.

Clears the perception bitmap and centers it on the car's position. For each lidar measurement, computes the global-to-perception window mapping for the detected point and draws a line from the car's center to that point. The color of the line depends on whether a cone was detected or not.

Parameters

<i>measures</i>	Pointer to an array of lidar measurements.
-----------------	--

Here is the caller graph for this function:



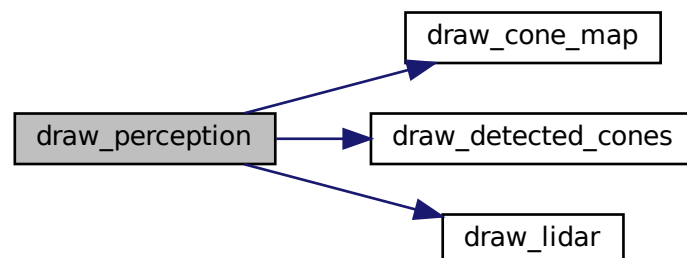
7.12.2.10 draw_perception()

```
void draw_perception ( )
```

Renders the complete perception layer.

Draws perception results.

Combines the lidar lines, detected cones, and cone map visualizations to create the perception overlay. Afterwards, the perception bitmap is blended onto the main display buffer offset in relation to the car's position. Here is the call graph for this function:



Here is the caller graph for this function:

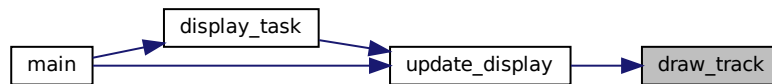


7.12.2.11 draw_track()

```
void draw_track ( )
```

Draws the track.

Checks that the track bitmap has valid dimensions, and if so, draws the track image onto the display buffer. If the bitmap dimensions are invalid, an error message is shown. Here is the caller graph for this function:



7.12.2.12 draw_trajectory()

```
void draw_trajectory (
    waypoint * trajectory )
```

Draws the trajectory of the car.

Draws the planned trajectory.

Clears and sets the background for the trajectory bitmap and iterates over each trajectory waypoint, drawing each as a filled circle. In DEBUG mode, the waypoint index is also displayed and an extra line is drawn from the car sprite center.

Parameters

<i>trajectory</i>	Pointer to an array of waypoint structures representing the car's trajectory.
-------------------	---

Here is the caller graph for this function:



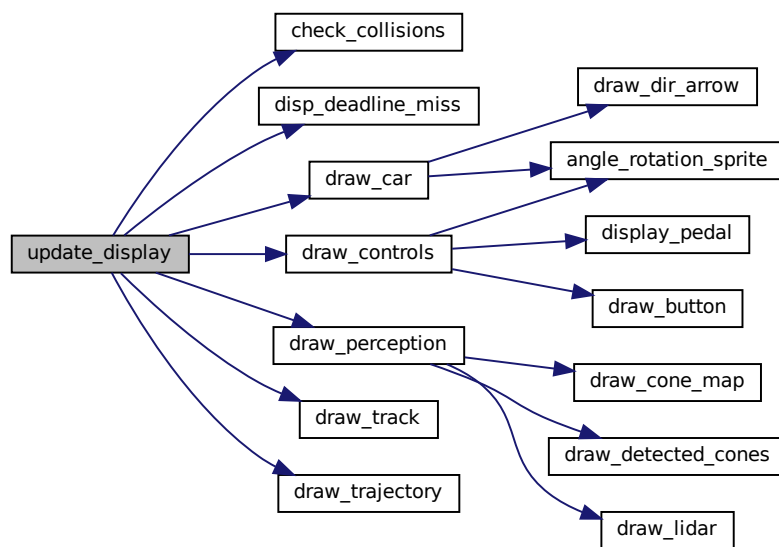
7.12.2.13 update_display()

```
void update_display ( )
```

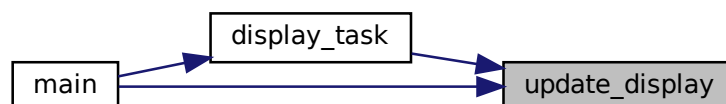
Updates the entire display.

Updates the display.

Locks the drawing mutex and sequentially renders the background, track, car, perception, trajectory, title text, and controls on the display buffer. Once all elements are rendered, the display buffer is blitted to the screen, and the mutex is subsequently unlocked. Here is the call graph for this function:



Here is the caller graph for this function:



7.12.3 Variable Documentation

7.12.3.1 btn_state_autonomous

```
int btn_state_autonomous = 0
```

7.12.3.2 btn_state_cones

```
int btn_state_cones = 0
```

7.12.3.3 btn_state_map

```
int btn_state_map = 1
```

7.12.3.4 btn_state_perception

```
int btn_state_perception = 1
```

7.12.3.5 btn_state_traj

```
int btn_state_traj = 1
```

7.12.3.6 buttons

```
Button buttons[num_buttons]
```

Initial value:

```
= {  
    {20, 20, 150, 30, "Clean track", &btn_state_cones},  
    {20, 60, 150, 30, "Lidar", &btn_state_perception},  
    {20, 100, 150, 30, "Mapping", &btn_state_map},  
    {20, 140, 150, 30, "Trajectory", &btn_state_traj},  
    {20, 180, 150, 30, "Full Autonomous", &btn_state_autonomous}  
}
```

Periodic perception task.

This task acquires LiDAR measurements, resets the array of detected cones, and performs mapping based on the current position and orientation of the vehicle. After processing the sensor data, it signals the trajectory planning task using a semaphore. The task executes continuously until the ESC key is pressed.

Parameters

<i>in</i>	<i>arg</i>	Pointer used to determine the task's index.
-----------	------------	---

Returns

Always returns NULL.

Note

The function uses a runtime measurement call to monitor its execution time.

Periodic trajectory planning task.

This task waits for a signal from the perception task to ensure updated sensor data is available. It then calculates a new trajectory for the vehicle based on its current position, orientation, and the detected cones from the environment. The task runs periodically and terminates when the ESC key is pressed.

Parameters

<i>in</i>	<i>arg</i>	Pointer used to determine the task's index.
-----------	------------	---

Returns

Always returns NULL.

Note

The task's operation is encapsulated between runtime measurement calls.

Periodic control task.

This task is responsible for controlling the vehicle's movement. It chooses between keyboard-based control and autonomous control based on the state of the 'A' key. If autonomous mode is active (i.e., 'A' is pressed), it uses the computed trajectory, otherwise it relies on user input to drive the vehicle. Execution continues until the ESC key is pressed.

Parameters

<i>in</i>	<i>arg</i>	Pointer used to determine the task's index.
-----------	------------	---

Returns

Always returns NULL.

Note

The function uses runtime measurement calls to monitor execution time per cycle.

Periodic display task.

This task updates the graphical display of the simulation environment. It refreshes the on-screen information such as the vehicle's position, sensor data, and other simulation elements. The task continues running until the ESC key is pressed.

Parameters

in	arg	Pointer used to determine the task's index.
----	-----	---

Returns

Always returns NULL.

Note

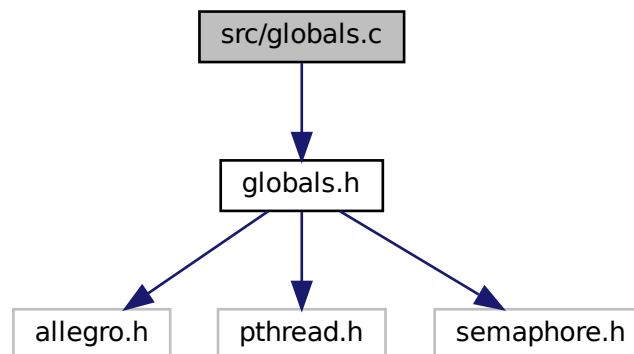
Like the other tasks, execution time is tracked using runtime measurement calls.

7.13 src/globals.c File Reference

Global definitions and state variables for the 2D FSAE simulation.

```
#include "globals.h"
```

Include dependency graph for globals.c:



Variables

- `const char * title = "2D FSAE sim by rimaturus"`
- `const int X_MAX = (19 * px_per_meter)`
- `const int Y_MAX = (10 * px_per_meter)`
- `const float cone_radius = 0.05f`
- `float car_x = 4.0f`

- float `car_y` = 3.0f
- int `car_angle` = 270
- cone cones [MAX_CONES_MAP]
- cone starting_cone_positions [MAX_CONES_MAP]
- BITMAP * `control_panel` = NULL
- BITMAP * `steering_wheel` = NULL
- BITMAP * `throttle_gauge` = NULL
- BITMAP * `background` = NULL
- BITMAP * `track` = NULL
- BITMAP * `car` = NULL
- BITMAP * `perception` = NULL
- BITMAP * `trajectory_bmp` = NULL
- BITMAP * `display_buffer` = NULL
- int `grass_green`
- int `asphalt_gray`
- int `white`
- int `pink`
- int `yellow`
- int `blue`
- int `red`
- int `green`
- `pointcloud_t` measures [360]
- `sem_t` `lidar_sem`
- `pthread_mutex_t` `draw_mutex` = PTHREAD_MUTEX_INITIALIZER
- `pthread_mutex_t` `map_mutex` = PTHREAD_MUTEX_INITIALIZER
- `pthread_mutex_t` `car_mutex` = PTHREAD_MUTEX_INITIALIZER
- int `dl_miss_perception` = 0
- int `dl_miss_trajectory` = 0
- int `dl_miss_control` = 0
- int `dl_miss_display` = 0
- int `dl_miss_settings` = 0

7.13.1 Detailed Description

Global definitions and state variables for the 2D FSAE simulation.

This file declares and initializes global constants, variables, and resources used by the simulation.

- Simulation Information:

—

7.13.2 Variable Documentation

7.13.2.1 asphalt_gray

```
int asphalt_gray
```

7.13.2.2 background

```
BITMAP* background = NULL
```

-

7.13.2.3 blue

```
int blue
```

7.13.2.4 car

```
BITMAP* car = NULL
```

-

7.13.2.5 car_angle

```
int car_angle = 270
```

The initial orientation angle of the car (in degrees).

- Graphical Resources:

—

7.13.2.6 car_mutex

```
pthread_mutex_t car_mutex = PTHREAD_MUTEX_INITIALIZER
```

7.13.2.7 car_x

```
float car_x = 4.0f
```

The initial X-coordinate position of the car within the simulation.

-

7.13.2.8 car_y

```
float car_y = 3.0f
```

7.13.2.9 cone_radius

```
const float cone_radius = 0.05f
```

The radius (in meters) for cone-related calculations, set to 0.05.

- Car Initial State:

-

7.13.2.10 cones

```
cone cones[MAX_CONES_MAP]
```

7.13.2.11 control_panel

```
BITMAP* control_panel = NULL
```

-

7.13.2.12 display_buffer

```
BITMAP* display_buffer = NULL
```

Bitmap pointers used for rendering various components of the simulation such as the background, track markings, car image, perception overlays, trajectory visualizations, and the overall display buffer.

- Color Definitions:

–

7.13.2.13 dl_miss_control

```
int dl_miss_control = 0
```

7.13.2.14 dl_miss_display

```
int dl_miss_display = 0
```

7.13.2.15 dl_miss_perception

```
int dl_miss_perception = 0
```

7.13.2.16 dl_miss_settings

```
int dl_miss_settings = 0
```

7.13.2.17 dl_miss_trajectory

```
int dl_miss_trajectory = 0
```

7.13.2.18 draw_mutex

```
pthread_mutex_t draw_mutex = PTHREAD_MUTEX_INITIALIZER
```

A POSIX mutex that ensures mutual exclusion during drawing operations.

Note

The multiplier 'px_per_meter' used for computing X_MAX and Y_MAX is defined elsewhere.

7.13.2.19 grass_green

```
int grass_green
```

7.13.2.20 green

```
int green
```

7.13.2.21 lidar_sem

```
sem_t lidar_sem
```

A POSIX semaphore used to synchronize access to LiDAR data.

-

7.13.2.22 map_mutex

```
pthread_mutex_t map_mutex = PTHREAD_MUTEX_INITIALIZER
```

7.13.2.23 measures

```
pointcloud_t measures[360]
```

An array of 360 LiDAR measurements (one per degree) representing the simulated point cloud data.

- Thread Synchronization:

-

7.13.2.24 perception

```
BITMAP* perception = NULL
```

-

7.13.2.25 pink

```
int pink
```

7.13.2.26 red

```
int red
```

7.13.2.27 starting_cone_positions

```
cone starting_cone_positions[MAX\_CONES\_MAP]
```

Store the default cone positions before collisions

7.13.2.28 steering_wheel

```
BITMAP* steering_wheel = NULL
```

-

7.13.2.29 throttle_gauge

```
BITMAP* throttle_gauge = NULL
```

Bitmap pointers for UI elements including the control panel, steering wheel, and throttle gauge.

-

7.13.2.30 title

```
const char* title = "2D FSAE sim by rimaturus"
```

The simulation title ("2D FSAE sim by rimaturus").

- Simulation Dimensions:

—

7.13.2.31 track

```
BITMAP* track = NULL
```

-

7.13.2.32 trajectory_bmp

```
BITMAP* trajectory_bmp = NULL
```

-

7.13.2.33 white

```
int white
```

7.13.2.34 X_MAX

```
const int X_MAX = (19 * px_per_meter)
```

The maximum horizontal dimension of the simulation in pixels, computed as $19 * \text{px_per_meter}$.

-

7.13.2.35 Y_MAX

```
const int Y_MAX = (10 * px_per_meter)
```

The maximum vertical dimension of the simulation in pixels, computed as $12 * px_per_meter$.

- Physics and Geometry:

—

7.13.2.36 yellow

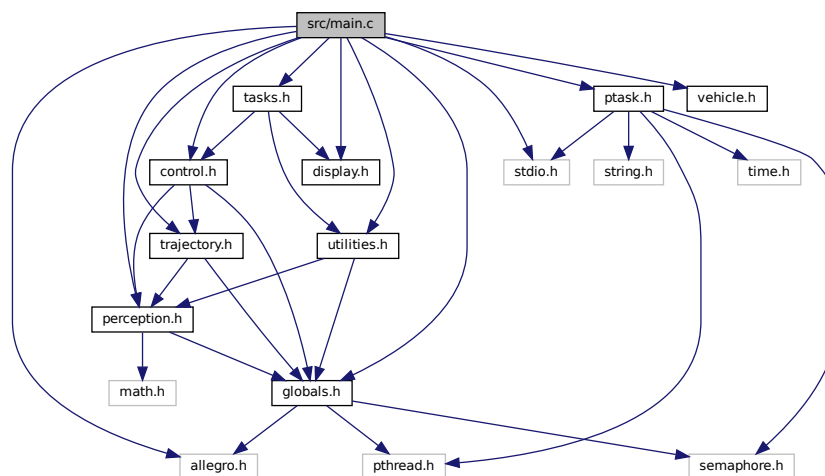
```
int yellow
```

7.14 src/main.c File Reference

Main implementation of the 2D FSAE Simulation.

```
#include <stdio.h>
#include <allegro.h>
#include "control.h"
#include "display.h"
#include "globals.h"
#include "perception.h"
#include "tasks.h"
#include "trajectory.h"
#include "utilities.h"
#include "vehicle.h"
#include "ptask.h"
#include "task.h"
```

Include dependency graph for main.c:



Functions

- void `init_allegro` ()
- void `init_track` ()
- void `init_car` ()
- void `init_perception` ()
- void `init_control_panel` ()
- void `init_bitmaps` ()
- int `main` (void)
- void `init_trajectory` ()

Variables

- const char `filename` [100] = "track/cones.yaml"
Application entry point.
- int `car_x_px`
- int `car_y_px`
- int `car_bitmap_x`
- int `car_bitmap_y`

7.14.1 Detailed Description

Main implementation of the 2D FSAE Simulation.

This file initializes the Allegro graphics library, sets up various bitmaps for simulation elements, and creates periodic tasks for perception, trajectory, control, and display. It also handles input, such as waiting for the user to press ESC to exit the simulation.

These routines configure the graphical environment, load assets (bitmaps), and initialize the simulation track including cones from an external YAML file. The program makes use of a periodic task system (using `SCHED_OTHER`) and employs mutexes to synchronize drawing operations.

7.14.2 Function Documentation

7.14.2.1 `init_allegro()`

```
void init_allegro ( )
```

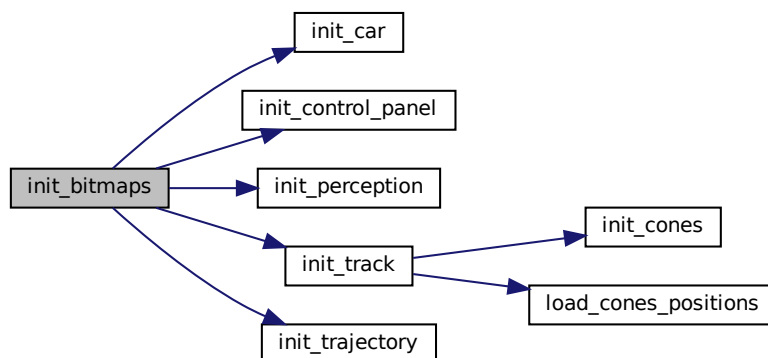
Here is the caller graph for this function:



7.14.2.2 init_bitmaps()

```
void init_bitmaps ( )
```

Here is the call graph for this function:



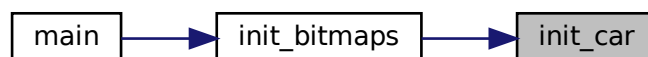
Here is the caller graph for this function:



7.14.2.3 init_car()

```
void init_car ( )
```

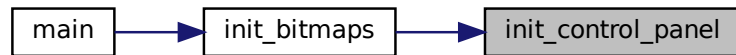
Here is the caller graph for this function:



7.14.2.4 init_control_panel()

```
void init_control_panel ( )
```

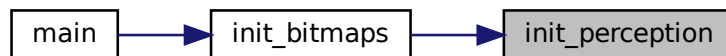
Here is the caller graph for this function:



7.14.2.5 init_perception()

```
void init_perception ( )
```

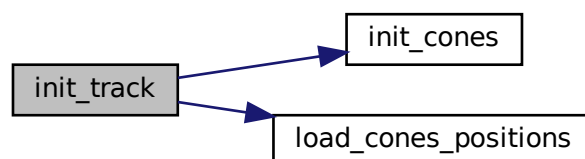
Here is the caller graph for this function:



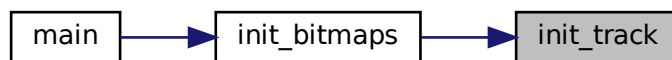
7.14.2.6 init_track()

```
void init_track ( )
```

Here is the call graph for this function:



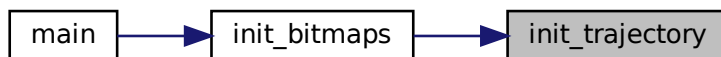
Here is the caller graph for this function:



7.14.2.7 init_trajectory()

```
void init_trajectory ( )
```

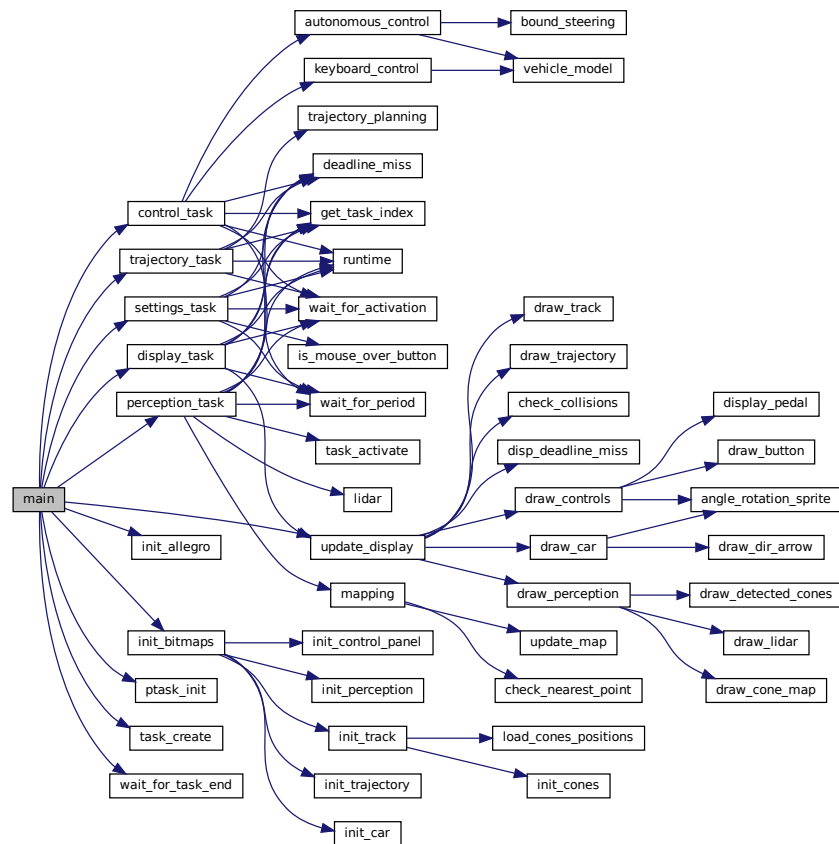
Here is the caller graph for this function:



7.14.2.8 main()

```
int main (
    void )
```

Here is the call graph for this function:



7.14.3 Variable Documentation

7.14.3.1 car_bitmap_x

```
int car_bitmap_x
```

7.14.3.2 car_bitmap_y

```
int car_bitmap_y
```

7.14.3.3 car_x_px

```
int car_x_px
```

7.14.3.4 `car_y_px`

```
int car_y_px
```

7.14.3.5 `filename`

```
const char filename[100] = "track/cones.yaml"
```

Application entry point.

The `main()` function initializes the graphics system, creates display bitmaps, and sets up periodic tasks for different simulation modules:

- Perception: processes sensor data to detect environment features.
- Trajectory: calculates the planned path for the vehicle.
- Control: manages vehicle control based on perception and trajectory.
- Display: updates the visual representation of the simulation.

After task creation, the main loop waits for all tasks to terminate (typically on ESC key press), cleans up the graphical buffers, and exits the simulation.

See also

[init_allegro\(\)](#), [init_bitmaps\(\)](#), [update_screen\(\)](#), [task_create\(\)](#), [wait_for_task_end\(\)](#)

Initializes the Allegro graphics system.

Sets up the graphics data structures, installs keyboard and mouse handlers, and configures the graphics mode. It defines the color depth and initializes several colors (e.g., grass green, asphalt gray, pink) used throughout the simulation. The display window is created with a title and switch mode, and finally the screen is cleared.

See also

[allegro_init\(\)](#), [install_keyboard\(\)](#), [install_mouse\(\)](#), [set_gfx_mode\(\)](#), [set_window_title\(\)](#), [clear_to_color\(\)](#)

Initializes the track bitmap.

Creates a bitmap representing the track and clears it to an asphalt color. It loads cone positions from an external YAML configuration file and plots track cones on the track bitmap. Only valid cones (with a valid color) are rendered, using a radius scaled to pixels per meter.

See also

[load_cones_positions\(\)](#), [init_cones\(\)](#), [clear_bitmap\(\)](#), [circlefill\(\)](#)

Initializes the car bitmap and positions.

Loads the car sprite bitmap from a file. It computes the car position in pixels, ensuring that the car's graphical representation is centered on its logical position by offsetting the bitmap's top-left coordinates.

See also

`load_bitmap()`, `clear_bitmap()`, `circlefill()`

Initializes the perception bitmap.

Creates a dedicated bitmap for representing the perceptual data (e.g., sensor outputs) of the simulation. The dimensions are based on the maximum range of the sensor (scaled by pixels per meter), ensuring that the bitmap covers the necessary area for sensor information. The bitmap is cleared and set to a transparent color.

See also

`create_bitmap()`, `clear_bitmap()`, `clear_to_color()`

Initializes the trajectory bitmap.

This function creates and configures the bitmap that visualizes the calculated trajectory path. The bitmap is cleared and its background is set transparent (using the designated pink color).

See also

`create_bitmap()`, `clear_bitmap()`, `clear_to_color()`

Initializes visual control elements.

Loads and initializes bitmaps for user visual controls such as the steering wheel (bitmap for steering) and a visual throttle gauge. In case of any loading error, the application exits with an error message.

See also

`load_bitmap()`, `create_bitmap()`, `clear_bitmap()`, `clear_to_color()`

Initializes all bitmaps used for displaying the simulation.

Creates the display buffer and background bitmaps, clearing them to the proper color. Then, it calls several initialization functions that prepare all simulation-specific bitmaps (track, car, perception, trajectory, and visual controls).

See also

`create_bitmap()`, [init_track\(\)](#), [init_car\(\)](#), [init_perception\(\)](#), [init_trajectory\(\)](#), `init_visual_controls()`

Updates the display screen.

Locks a mutex to prevent concurrent drawing modifications, clears the display buffer, and performs a series of drawing operations:

- Renders the steering wheel with rotation and scaling.
- Draws the background and track bitmaps.
- Rotates, scales, and draws the car sprite ensuring correct orientation.
- Draws the perception and trajectory bitmaps. Finally, the updated display buffer is blitted to the actual screen, and the mutex is unlocked.

See also

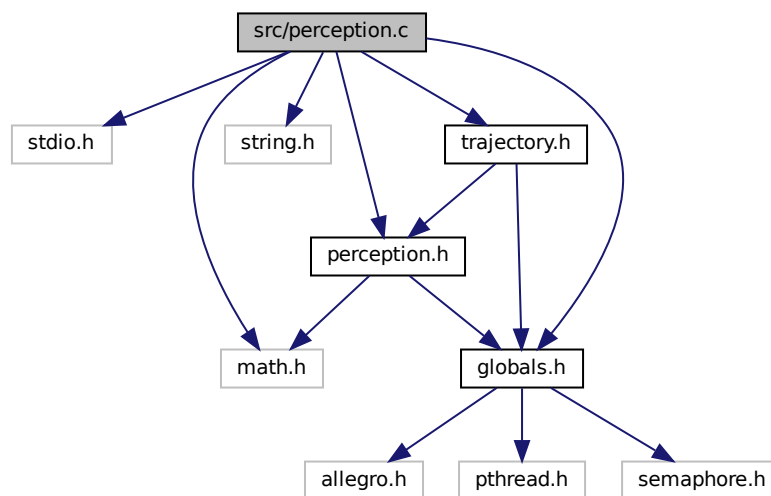
`pthread_mutex_lock()`, `clear_to_color()`, `rotate_scaled_sprite()`, `draw_sprite()`, `blit()`, `pthread_mutex_unlock()`

7.15 src/perception.c File Reference

Implements LiDAR-based cone detection and mapping using circle Hough transform.

```
#include <stdio.h>
#include <math.h>
#include <string.h>
#include "globals.h"
#include "perception.h"
#include "trajectory.h"
```

Include dependency graph for perception.c:



Functions

- void [check_nearest_point](#) (int angle, float new_point_x, float new_point_y, int color, [cone_border](#) *cone_↔ borders)
- void [lidar](#) ([pointcloud_t](#) *measures)
- void [init_cone_borders](#) ([cone_border](#) *cone_borders)
- void [calculate_circle_points](#) (float center_x, float center_y, int color, [cone](#) *circle_points)
- void [find_closest_points](#) ([Hough_circle_point_t](#) *circumference_points, float point_x, float point_↔ y, [Hough_circle_point_t](#) *reference_points, int ref_size)
- void [find_local_minima](#) ([Hough_circle_point_t](#) *points, int *first_min, int *second_min)
- float * [find_cone_center](#) ([Hough_circle_point_t](#) *possible_centers, int center_count)
- void [mapping](#) ([cone](#) *detected_cones)
- void [update_map](#) ([cone](#) *detected_cones)

Variables

- const int [sliding_window](#) = 360
- const int [angle_step](#) = 1
- int [start_angle](#) = 0
- const float [ignore_distance](#) = 0.5f

- const float `distance_resolution` = 0.01f
- cone `detected_cones` [`MAX_DETECTED_CONES`]
- int `n_candidates` = 0
- candidate_cone `candidates` [`MAX_CANDIDATES`]
- int `track_map_idx` = 0
- cone `track_map` [`MAX_CONES_MAP`]

7.15.1 Detailed Description

Implements LiDAR-based cone detection and mapping using circle Hough transform.

This file contains functions to process LiDAR measurements, cluster points belonging to the same cone, compute cone centers with a Hough circle parameterization, and update a global map of detected cones.

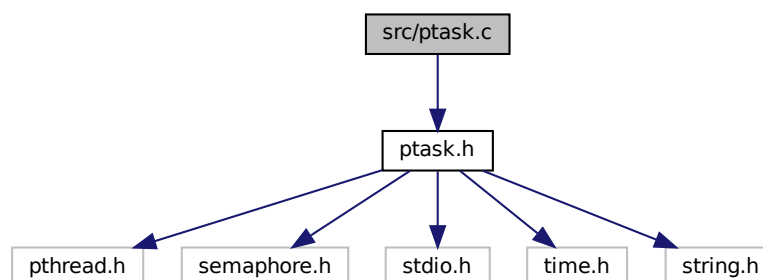
Global variables:

- `sliding_window`: Number of angles scanned (360 degrees).
- `angle_step`: Step in degrees for each LiDAR scan.
- `start_angle`: Starting angle of the LiDAR scan.
- `ignore_distance`: Minimum distance threshold for detection.
- `distance_resolution`: Increment step for distances along LiDAR rays.
- `detected_cones`: Array that holds final detected cone positions and colors.
- `n_candidates`: Number of candidate cones currently tracked.
- `candidates`: Array that holds candidate cone detections.
- `track_map_idx`: Index for the global map where detected cones are stored.
- `track_map`: Global map of cone positions and colors.

7.16 src/ptask.c File Reference

```
#include "ptask.h"
```

Include dependency graph for ptask.c:



Macros

- `#define` [PTASK_IMPLEMENTATION](#)

7.16.1 Macro Definition Documentation

7.16.1.1 PTASK_IMPLEMENTATION

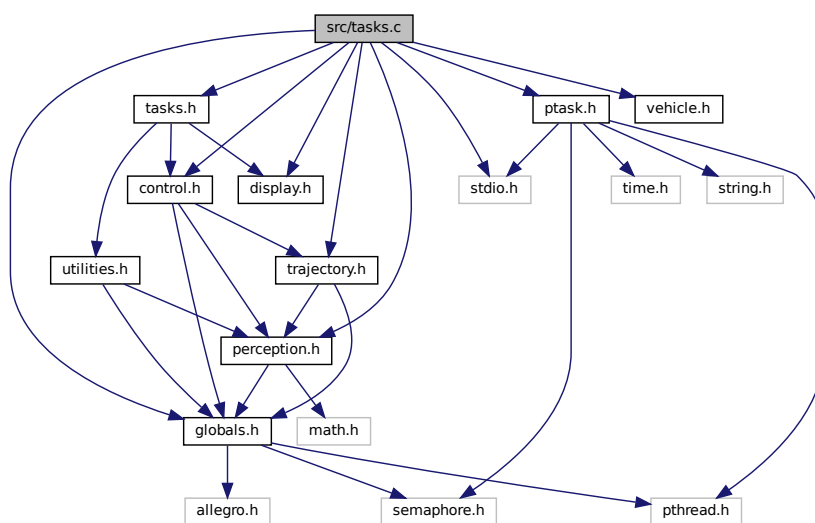
```
#define PTASK_IMPLEMENTATION
```

7.17 src/tasks.c File Reference

Contains periodic task functions for LiDAR perception, trajectory planning, vehicle control, and display update in a 2D simulation environment.

```
#include <stdio.h>
#include "globals.h"
#include "tasks.h"
#include "perception.h"
#include "trajectory.h"
#include "vehicle.h"
#include "display.h"
#include "ptask.h"
#include "control.h"
```

Include dependency graph for tasks.c:



Functions

- void * [perception_task](#) (void *arg)
Perception task.
- void * [trajectory_task](#) (void *arg)
- void * [control_task](#) (void *arg)
- void * [display_task](#) (void *arg)
- int [is_mouse_over_button](#) ([Button](#) *btn, int mouse_x, int mouse_y)
- void * [settings_task](#) (void *arg)

Variables

- [Button](#) [buttons](#) []
Periodic perception task.
- int [btn_state_cones](#)
- int [btn_state_perception](#)
- int [btn_state_map](#)
- int [btn_state_traj](#)
- int [btn_state_autonomous](#)

7.17.1 Detailed Description

Contains periodic task functions for LiDAR perception, trajectory planning, vehicle control, and display update in a 2D simulation environment.

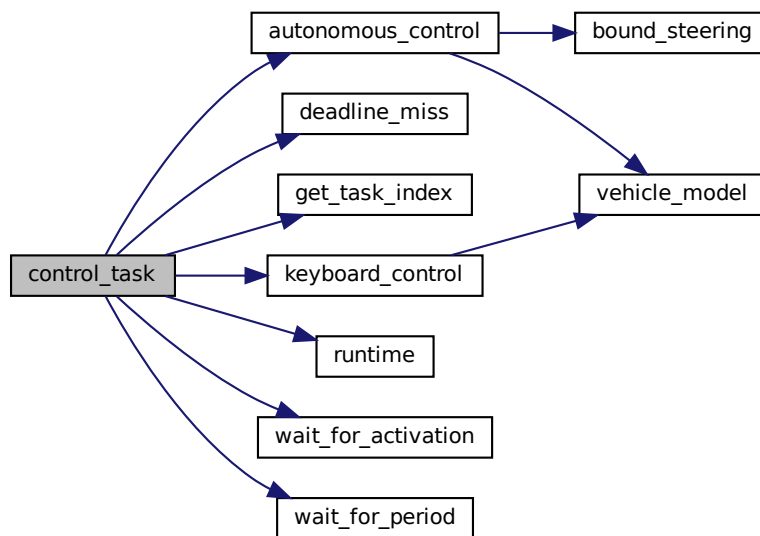
This file implements tasks that run periodically in separate threads. Each task is synchronized with others through semaphores and managed by a periodic task scheduler.

7.17.2 Function Documentation

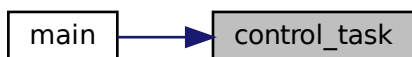
7.17.2.1 control_task()

```
void* control_task (  
    void * arg )
```

Here is the call graph for this function:



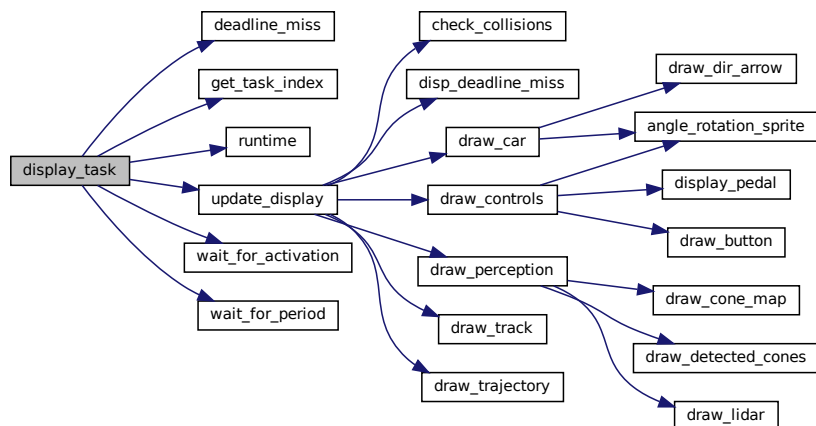
Here is the caller graph for this function:



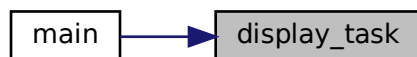
7.17.2.2 display_task()

```
void* display_task (  
    void * arg )
```

Here is the call graph for this function:



Here is the caller graph for this function:



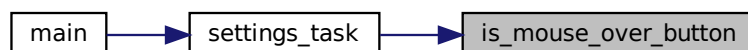
7.17.2.3 is_mouse_over_button()

```

int is_mouse_over_button (
    Button * btn,
    int mouse_x,
    int mouse_y )

```

Here is the caller graph for this function:



7.17.2.4 perception_task()

```
void* perception_task (  
    void * arg )
```

Perception task.

This function executes perception related operations, such as processing sensor data or simulation input to understand the surrounding environment.

Parameters

<i>arg</i>	Pointer to the arguments required by the task. The actual type and usage of this parameter are defined by the implementation context.
------------	---

Returns

Pointer representing the function's outcome, typically not used.

Trajectory planning task.

This function is responsible for calculating and updating the movement trajectory for the entity in simulation. It handles the logic for path planning based on environmental data and simulation dynamics.

Parameters

<i>arg</i>	Pointer to the arguments specific to the trajectory task.
------------	---

Returns

Pointer representing the result of the function, usually unused.

Control task.

This function manages the control logic, determining inputs for either keyboard control or autonomous behavior. It integrates various control signals and decisions, ensuring the entity reacts appropriately within the simulation.

Parameters

<i>arg</i>	Pointer to a structure or value containing control parameters.
------------	--

Returns

Pointer to the result of the control processing, typically not utilized.

Display update task.

This function handles the rendering process for the simulation's trajectory display. It updates the visual output based on the current state of the simulation data.

Parameters

<i>arg</i>	Pointer to display specific parameters or context information.
------------	--

Returns

Pointer indicating the outcome of the display operation, generally unused.

Task managing simulation settings and configuration

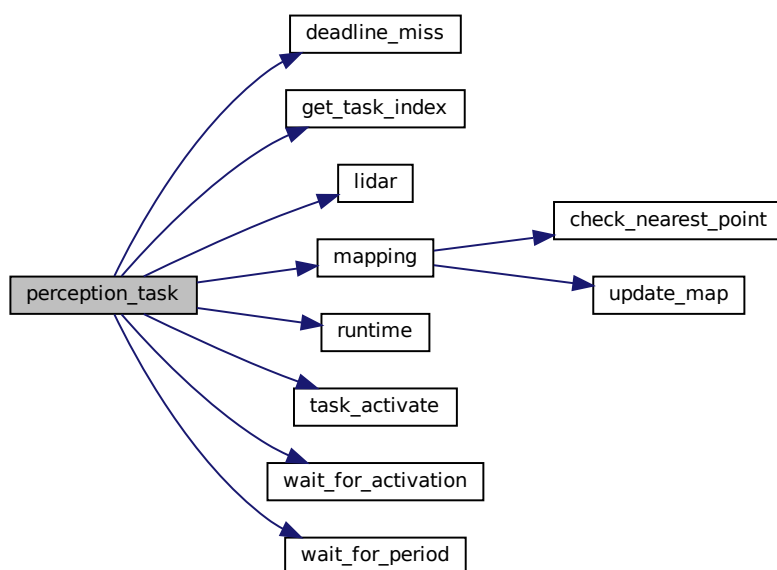
Parameters

<i>arg</i>	Pointer to task arguments (void pointer for thread compatibility)
------------	---

Returns

void* Return value of the thread (unused)

Thread task that handles simulation settings and configuration updates. Should be used with `pthread_create()`. Here is the call graph for this function:



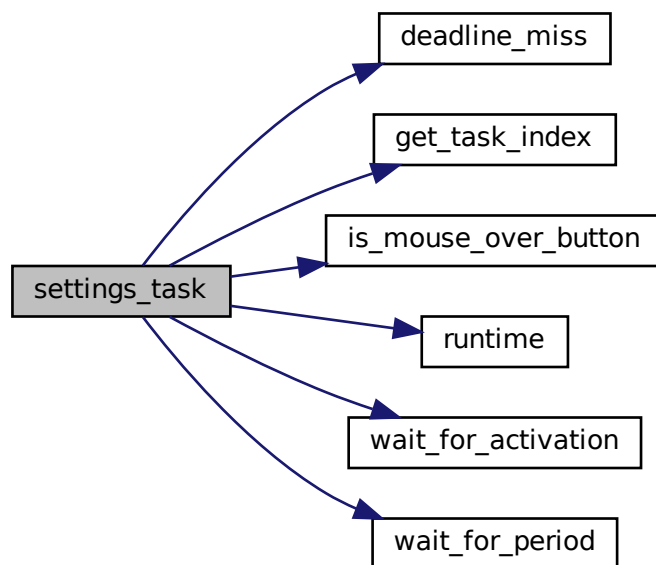
Here is the caller graph for this function:



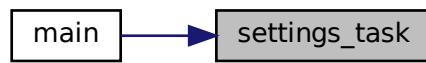
7.17.2.5 settings_task()

```
void* settings_task (  
    void * arg )
```

Here is the call graph for this function:



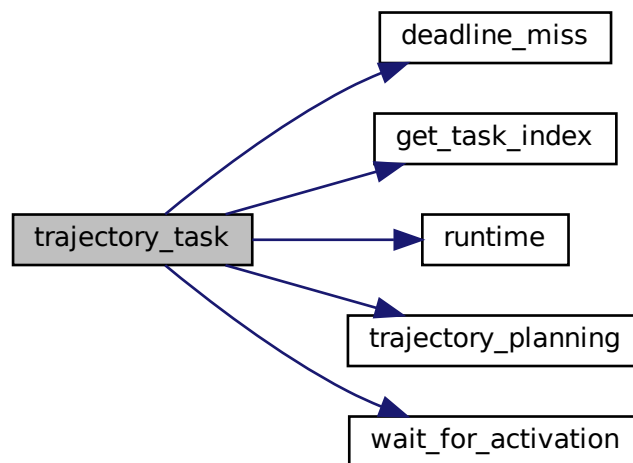
Here is the caller graph for this function:



7.17.2.6 trajectory_task()

```
void* trajectory_task (  
    void * arg )
```

Here is the call graph for this function:



Here is the caller graph for this function:



7.17.3 Variable Documentation

7.17.3.1 btn_state_autonomous

```
int btn_state_autonomous [extern]
```

7.17.3.2 btn_state_cones

```
int btn_state_cones [extern]
```

7.17.3.3 btn_state_map

```
int btn_state_map [extern]
```

7.17.3.4 btn_state_perception

```
int btn_state_perception [extern]
```

7.17.3.5 btn_state_traj

```
int btn_state_traj [extern]
```

7.17.3.6 buttons

```
Button buttons[] [extern]
```

Periodic perception task.

This task acquires LiDAR measurements, resets the array of detected cones, and performs mapping based on the current position and orientation of the vehicle. After processing the sensor data, it signals the trajectory planning task using a semaphore. The task executes continuously until the ESC key is pressed.

Parameters

in	arg	Pointer used to determine the task's index.
----	-----	---

Returns

Always returns NULL.

Note

The function uses a runtime measurement call to monitor its execution time.

Periodic trajectory planning task.

This task waits for a signal from the perception task to ensure updated sensor data is available. It then calculates a new trajectory for the vehicle based on its current position, orientation, and the detected cones from the environment. The task runs periodically and terminates when the ESC key is pressed.

Parameters

in	arg	Pointer used to determine the task's index.
----	-----	---

Returns

Always returns NULL.

Note

The task's operation is encapsulated between runtime measurement calls.

Periodic control task.

This task is responsible for controlling the vehicle's movement. It chooses between keyboard-based control and autonomous control based on the state of the 'A' key. If autonomous mode is active (i.e., 'A' is pressed), it uses the computed trajectory, otherwise it relies on user input to drive the vehicle. Execution continues until the ESC key is pressed.

Parameters

in	arg	Pointer used to determine the task's index.
----	-----	---

Returns

Always returns NULL.

Note

The function uses runtime measurement calls to monitor execution time per cycle.

Periodic display task.

This task updates the graphical display of the simulation environment. It refreshes the on-screen information such as the vehicle's position, sensor data, and other simulation elements. The task continues running until the ESC key is pressed.

Parameters

<code>in</code>	<code>arg</code>	Pointer used to determine the task's index.
-----------------	------------------	---

Returns

Always returns NULL.

Note

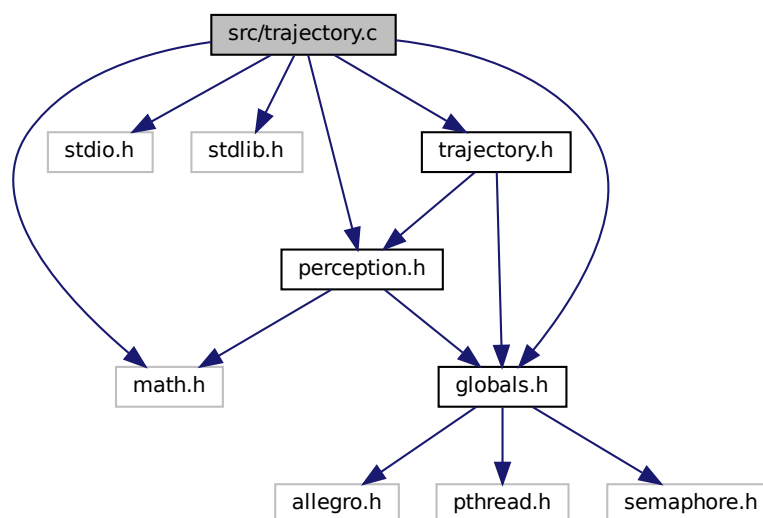
Like the other tasks, execution time is tracked using runtime measurement calls.

7.18 src/trajectory.c File Reference

Implements trajectory planning based on detected cones.

```
#include <math.h>
#include <stdio.h>
#include <stdlib.h>
#include "trajectory.h"
#include "globals.h"
#include "perception.h"
```

Include dependency graph for trajectory.c:



Functions

- void `trajectory_planning` (`waypoint *trajectory`)

Variables

- int `trajectory_idx` = 0
Global index used to track the number of waypoints currently stored in the trajectory.
- `waypoint trajectory` [`MAX_WAYPOINTS`]
Global array of waypoints forming the planned trajectory.

7.18.1 Detailed Description

Implements trajectory planning based on detected cones.

This file contains the functionality to plan a driving trajectory using cone detection data. The trajectory is generated by connecting a global track map of cones (both blue and yellow) based on their proximity, and then finding midpoints between connected cones to serve as trajectory waypoints. Optionally, in debug builds, the cone connections are drawn for visualization.

Global Variables:

- `trajectory_idx`: An integer index representing the number of valid points in the generated trajectory.
- `trajectory`: An array of waypoints that stores the planned trajectory.

The trajectory planning process includes:

- Initializing trajectory points with invalid default values.
- Validating if there are enough cones in the track map to perform planning.
- Computing nearest neighbors for each cone (one for each color) for connection building.
- Drawing connection lines between cones when debugging is enabled.
- Generating trajectory waypoints by calculating the midpoint between connected cones.
- Reordering trajectory points based on proximity to ensure a smooth and sequential path.

Parameters

<code>trajectory</code>	Pointer to an array of waypoints where the computed trajectory will be stored.
-------------------------	--

Note

- The function assumes that there exists a correctly maintained global track map (`track_map`) and its size (`track_map_idx`).
- Debug drawing of cone connections is enclosed in an `#ifdef DEBUG` block.
- The maximum number of trajectory points is defined by `MAX_DETECTED_CONES`.

Warning

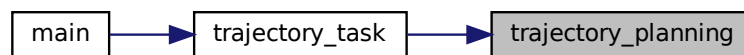
Ensure proper initialization of global variables and suitable allocation of the trajectory array before invoking this function.

7.18.2 Function Documentation

7.18.2.1 trajectory_planning()

```
void trajectory_planning (
    waypoint * trajectory )
```

Here is the caller graph for this function:



7.18.3 Variable Documentation

7.18.3.1 trajectory

```
waypoint trajectory[MAX_WAYPOINTS]
```

Global array of waypoints forming the planned trajectory.

This array stores the computed trajectory based on the detected cones. It is sized to accommodate up to twice the number of maximum detectable cones ($2 * \text{MAX_DETECTED_CONES}$).

7.18.3.2 trajectory_idx

```
int trajectory_idx = 0
```

Global index used to track the number of waypoints currently stored in the trajectory.

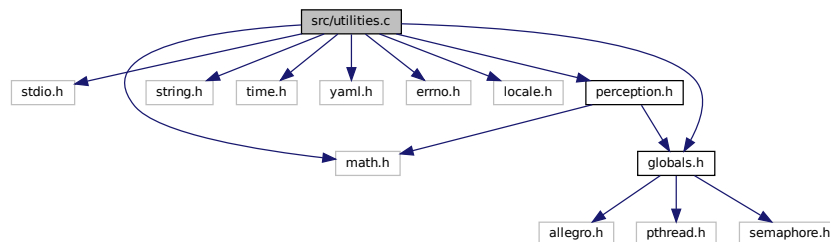
This variable indicates the current position in the trajectory array, facilitating the addition of new waypoints.

7.19 src/utilities.c File Reference

Utility functions for cone initialization, YAML cone loading, angle rotation conversion, and runtime performance measurement.

```
#include <stdio.h>
#include <math.h>
#include <string.h>
#include <time.h>
#include <yaml.h>
#include <errno.h>
#include <locale.h>
#include "globals.h"
#include "perception.h"
```

Include dependency graph for utilities.c:



Macros

- `#define _POSIX_C_SOURCE 199309L`
Initializes an array of cone structures.

Functions

- void `init_cones` (cone *cones)
Initializes an array of cone structures.
- void `load_cones_positions` (const char *filename, cone *cones, int max_cones)
- float `angle_rotation_sprite` (float angle)
- void `runtime` (int stop_signal, char *task_name)

Variables

- volatile float `offset_x` = 0.5f
- volatile float `offset_y` = 0.5f

7.19.1 Detailed Description

Utility functions for cone initialization, YAML cone loading, angle rotation conversion, and runtime performance measurement.

This file provides utility functions that include:

- Initializing an array of cone structures.
- Parsing a YAML file to load cone positions and colors.
- Converting an angle into a sprite rotation value.
- Timing code performance using the POSIX clock.

7.19.2 Macro Definition Documentation

7.19.2.1 `_POSIX_C_SOURCE`

```
#define _POSIX_C_SOURCE 199309L
```

Initializes an array of cone structures.

This function initializes each cone in the provided array by setting its x and y coordinates to 0.0f and its color to -1.

Parameters

<i>cones</i>	Pointer to the array of cone structures to be initialized.
--------------	--

Loads cone positions and colors from a YAML file.

This function parses the specified YAML file to extract cone position (x and y) and color data. The YAML file is expected to have a top-level key "cones" containing a sequence of cone mappings. Each cone mapping should provide:

- "x": A string representing the x coordinate (converted to float and scaled).
- "y": A string representing the y coordinate (converted to float and scaled).
- "color": A string representing the cone color (e.g., "yellow" or "blue").

The function prints out diagnostic messages to the standard output regarding the file being loaded, and reports any errors encountered during file opening or YAML parsing.

Parameters

<i>filename</i>	Path to the YAML file containing cone data.
<i>cones</i>	Pointer to the array of cone structures to be populated.
<i>max_cones</i>	Maximum number of cones to load into the array.

Converts an angle (in degrees) to a sprite rotation value.

This helper function transforms an input angle (in degrees) into a corresponding sprite rotation value. The conversion is based on the formula: $\text{sprite_rotation} = 64.0f - (128.0f * \text{angle} / 180.0f)$.

Parameters

<i>angle</i>	Angle in degrees.
--------------	-------------------

Returns

Sprite rotation value as a floating-point number.

Measures code performance based on a runtime signal.

This function is a helper to measure code execution performance if the PROFILING macro is defined. When called with a start signal (`stop_signal = 0`), it records the current time and prints a "START" message. When called with a stop signal (`stop_signal != 0`), it calculates the difference from the start time, prints an "END" message, and optionally prints the elapsed runtime in microseconds.

Note

This function depends on the POSIX clock `_gettime()` function and is active only if PROFILING is defined.

Parameters

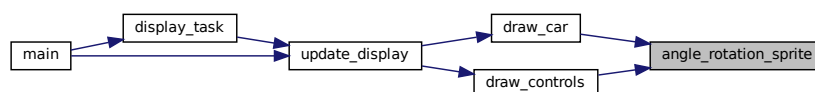
<i>stop_signal</i>	A flag indicating whether to start (0) or stop (non-zero) the runtime measurement.
<i>task_name</i>	A string representing the name of the task for which the runtime is being measured.

7.19.3 Function Documentation

7.19.3.1 angle_rotation_sprite()

```
float angle_rotation_sprite (
    float angle )
```

Here is the caller graph for this function:



7.19.3.2 init_cones()

```
void init_cones (
    cone * cones )
```

Initializes an array of cone structures.

This function sets up the specified array of cones so that each cone is properly initialized.

Parameters

<i>cones</i>	Pointer to the first element of an array of cone structures.
--------------	--

Loads cone positions from a file.

Reads cone position data from the provided file and populates the cone array accordingly. Only up to `max_cones` entries are loaded, ensuring the array bounds are respected.

Parameters

<i>filename</i>	Path to the file containing cone position data.
<i>cones</i>	Pointer to an array of cone structures where the positions will be stored.
<i>max_cones</i>	The maximum number of cones to load from the file.

Computes the sprite rotation angle.

Calculates and returns a new rotation angle for a sprite based on the input `angle`. The calculation may involve normalization or other transformations suitable for sprite handling.

Parameters

<i>angle</i>	The initial angle value (in degrees or radians, as defined by the application's convention).
--------------	--

Returns

The adjusted angle after applying the rotation transformation.

Handles runtime processes based on a stop signal.

Executes runtime tasks, potentially including logging or task management, until a stop condition defined by `stop_signal` is met. The associated task description is provided via `task_name`.

Parameters

<i>stop_signal</i>	Signal value indicating when to terminate the runtime process.
<i>task_name</i>	A string identifier for the task being executed.

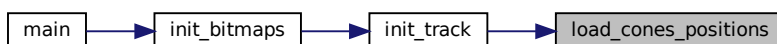
Here is the caller graph for this function:



7.19.3.3 load_cones_positions()

```
void load_cones_positions (
    const char * filename,
    cone * cones,
    int max_cones )
```

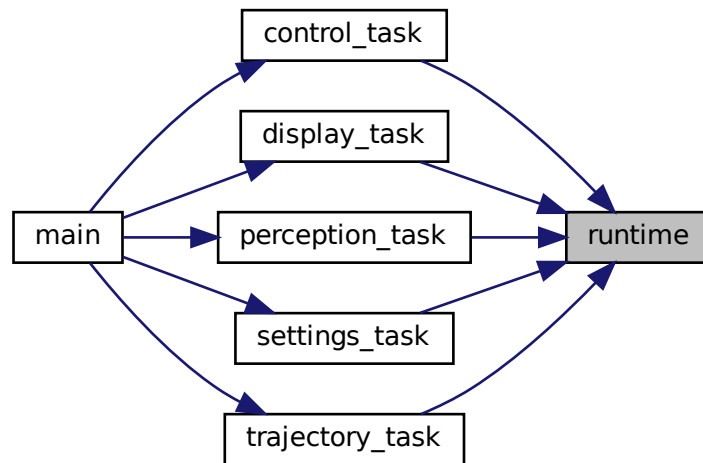
Here is the caller graph for this function:



7.19.3.4 runtime()

```
void runtime (
    int stop_signal,
    char * task_name )
```

Here is the caller graph for this function:



7.19.4 Variable Documentation

7.19.4.1 offset_x

```
volatile float offset_x = 0.5f
```

7.19.4.2 offset_y

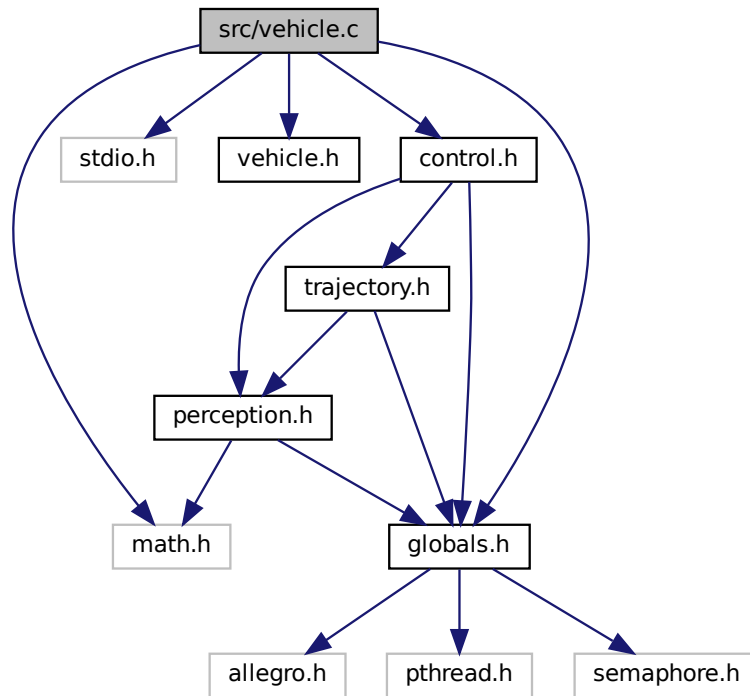
```
volatile float offset_y = 0.5f
```

7.20 src/vehicle.c File Reference

```
#include <math.h>
#include <stdio.h>
#include "vehicle.h"
#include "globals.h"
```

```
#include "control.h"
```

Include dependency graph for vehicle.c:



Macros

- `#define M_PI 3.14159265358979323846`
Updates the vehicle's position and orientation based on pedal and steering inputs.
- `#define CINEMATIC_MODEL`

Functions

- `void vehicle_model (float pedal, float target_steering)`
Simulates the vehicle's movement by updating its position and orientation.
- `void check_collisions ()`

7.20.1 Macro Definition Documentation

7.20.1.1 CINEMATIC_MODEL

```
#define CINEMATIC_MODEL
```

